AAICBI
AFRICA'S AI CAPACITY BUILDING INITIATIVE

VERSION 1.0 · 20 OCTOBER 2026

FOR LEADERSHIP, PARTNERS AND TECHNICAL REVIEWERS

Technology and Architecture Guide

The tools behind the platform, how the Game Studio, Certificate Studio and every engine work, and why each choice was made

Africa's AI Capacity Building Initiative (AAICBI)
Organised by Futybills Tech Community
Document version 1.0 · Published 20 October 2026
Prepared from the platform's source code, documentation and tests. Where a reason is written in the code it is quoted; where it is not, the guide says so.

Contents

1	About this guide	3
2	Architecture and design principles	6
3	The core platform stack	9
4	Identity, security and the permission engine	13
5	The Game Studio	14
6	The Ayakbi Arena, real time and the growth games	29
7	The Certificate Studio and document engines	36
8	The learning engines	44
9	The AI layer	48
10	Search, social, messaging, money and speed	53
11	Quality engineering and deployment	57
12	The decision register	60
13	Open items and honest limits	64
	Glossary	66
	Open items and how to verify this guide	67
	Document information and revision history	68

Tap or click any entry to jump to it. Each decision is labelled **Written rationale** (the reason is recorded in the code or documents), **Engineering inference** (our reading of why it fits) or **No written rationale**.

1 About this guide

What this document covers, how each claim was checked, and how to read the reasons given for each tool.

Purpose

The AAICBI Ecosystem is a large, real platform: learning management, an examination engine, a Game Studio, a Certificate Studio, a real-time game arena, a social feed, search, payments, messaging and several AI assistants. This guide lists the technology tools used to build it, explains what each tool does inside the platform, and explains **why it is a good solution** for AAICBI's situation — a small team serving learners across Africa, many on phones and slow connections, with limited operations staff.

It pays special attention to the **Game Studio**, the **Certificate Studio** and the platform's **engines**: the exam engine, assignment engine, course engine, guide engine, search engine, feed and moderation engine, notification engine, payment engine and permission engine.

How the reasons are labelled

A document like this can easily drift into marketing. To avoid that, every reason for a decision carries one of three labels:

WRITTEN RATIONALE
The reason is recorded by the engineers in the platform's own code comments or documents. We quote or paraphrase it and name the file.

ENGINEERING INFERENCE
No reason is written down, but the choice clearly fits the platform's needs. We say what we think the reason is. This is our reading, not the engineers' words.

NO WRITTEN RATIONALE
Nothing is written and we do not want to guess further. The tool is listed and its role described only.

How it was checked

- Every number here (pages, tables, tests, thresholds) was read from the repository: the application code, `prisma/schema.prisma`, `package.json`, the documents in `docs/`, the automated tests in `tests/` and the browser checks in `scripts/device-checks/`.
- Behaviour is described from the code. The guide did not load-test the platform and does not claim performance numbers that the repository does not contain.
- Where documents and code disagree, the code is treated as correct and the difference is listed in the final chapter.

- External services that have not been tested against the real provider (for example WhatsApp through Meta, and Google Translate) are marked as such.

The platform at a glance

Measure	Value (from the repository)
Web pages	220
Server endpoints (API route handlers)	482
Database tables (Prisma models) and enumerations	178 and 69
Database migrations (steps of change history)	109
Automated test files	144 (about 1,360 test cases), plus 111 browser and device check scripts
Files that use the shared error wrapper	439
Pure “core” logic files kept apart from database code	55
Languages for the interface	English, Yoruba, Igbo, Hausa, French and Nigerian Pidgin
Account types	Trainee, training organisation, employer, investor, startup founder; staff roles Super Admin, Admin, Instructor
Scheduled jobs	5 (daily access sweep, analytics alerts, weekly analytics report, arena sweep, hourly growth messages)

The technology tools in one table

Layer	Tool	What it does in AAICBI
Application framework	Next.js 14 (App Router) and React 18	Pages, server rendering and every server endpoint in one codebase.
Language	TypeScript (strict mode)	Typed code across pages, endpoints and database access.
Styling	Tailwind CSS, CSS variables, Fraunces and Manrope fonts	Brand colours, dark and light themes, readable type.

Layer	Tool	What it does in AAICBI
Database	PostgreSQL 16 with Prisma 5; pgvector and pg_trgm extensions	All data, migrations, duplicate detection for questions and fuzzy search indexes.
Hosting	Vercel (hosting, cron, Blob file storage) with Neon (managed Postgres)	Deployment, scheduled jobs, uploaded files.
Identity and security	jose (signed session tokens), bcryptjs, zod, Postgres-backed rate limiter	Sign-in, passwords, input validation, abuse limits.
AI	Anthropic Claude (Sonnet, Haiku, Opus) through the official SDK; Voyage AI for embeddings	Question extraction, grading help, feedback, assistants, content review.
Real time	Ably (server only) with a polling fallback	Live two-player games and chat updates.
Email, push and chat messages	Resend; web-push (VAPID) with a service worker; WhatsApp Cloud API	Notifications and reminders.
Payments	Paystack (naira), plain HTTP integration	Course payments, subscriptions, organisation fees, revenue split.
Documents and media	@react-pdf/renderer, mammoth, word-extractor, unpdf, jszip, node-html-parser, qrcode, sharp, next/og	Reports, certificates' QR, Word and PDF import, picture shrinking, result cards.
Certificate editor	Fabric.js 7	The drag-and-drop canvas of the Certificate Studio.
Charts and icons	Recharts, lucide-react	Analytics charts and the interface icon set.
Quality	Vitest, Playwright, axe-core	Unit tests, real-browser checks and accessibility sweeps.

2 Architecture and design principles

The shape of the system, and the ten habits in the code that make it dependable.

The system in one picture

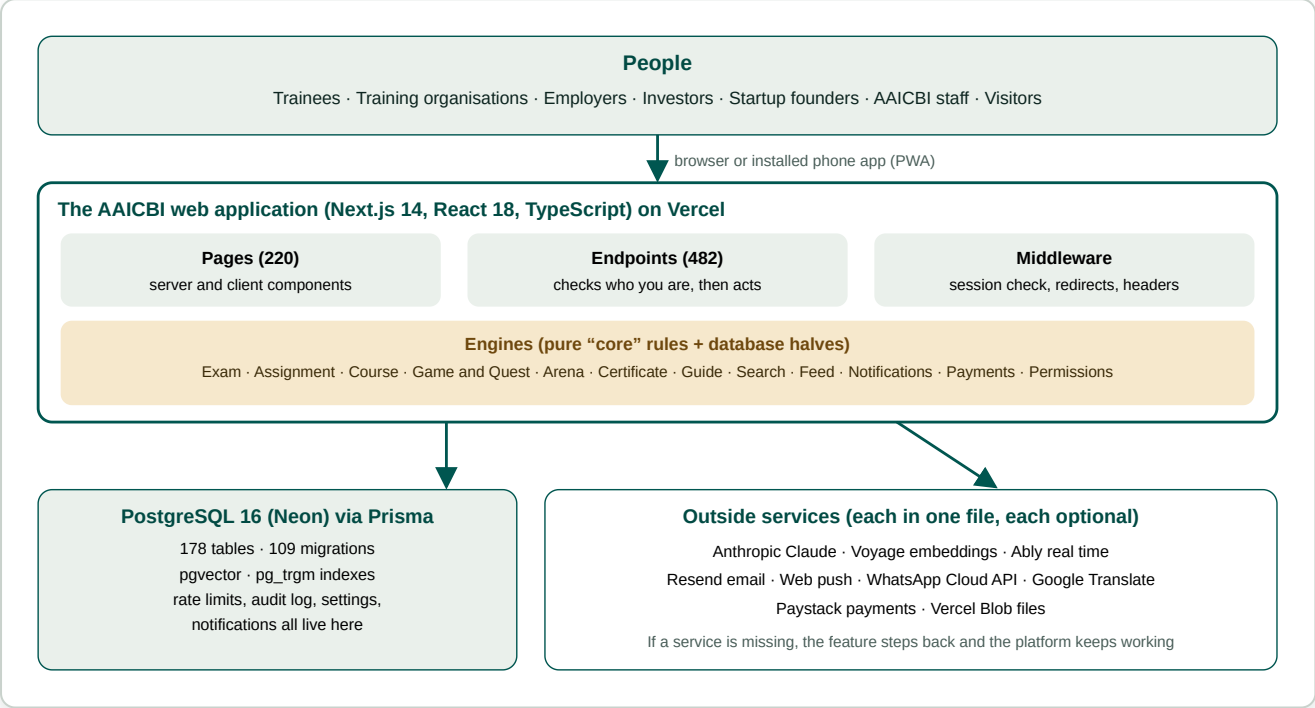


Figure 1. How the pieces fit. The database is the only infrastructure the platform cannot run without; every outside service is optional and isolated.

Ten design habits that show up across the whole codebase

1. Pure "core" rules, separate from database code

Each engine keeps its rules in a file that never touches the database (for example `examEngineCore.ts`, `rateLimitCore.ts`, `embeddingsCore.ts`, `quest/core.ts`, `arena/*Core.ts`) and a second file that reads and writes data. There are 55 such core files. This lets the rules be tested in milliseconds without a database and tuned in one place.

> **WRITTEN RATIONALE**
> The split came from a real failure: a test that imported a pure function pulled in a database client that could not be generated in that environment. The headers of `rateLimitCore.ts` and `embeddingsCore.ts` say so. `quest/core.ts` adds that every number of the learning design "can be tested and tuned in one place".

2. The server is the authority; the browser is never trusted

Timers, scores, answer keys, permissions and ownership are decided on the server. Examples: the exam clock is the server clock; answer keys are never sent while a test is in progress; an organisation's identity comes from its session, never from a request field.

> **WRITTEN RATIONALE**
> `examEngine.ts` : "A student changing their device clock cannot extend their own time." `rbac/guard.ts` : "The client is never trusted for who someone is, which organization they belong to, or what they may do."

3. Deny by default

Permissions start empty. A person can do something only if a rule grants it, and permissions are read from the database on every request, so a revoked permission applies to an open session immediately.

> **WRITTEN RATIONALE**
> `docs/rbac-audit-and-plan.md` : "Deny by default"; "a revoked permission applies to existing sessions at once. Nothing is cached across requests."

4. Every outside service is optional and isolated

Email, push, real time, embeddings, WhatsApp, translation, file storage and the AI provider each live behind one small file. If a key is missing, that feature quietly steps back and the rest keeps working. Only two settings are strictly required to run the platform: the signing secret and the AI key (for AI features).

> **WRITTEN RATIONALE**
> `notifications/email.ts` : a broken registration or password reset "because an email provider hiccupped" is worse than a skipped email. `arena/realtime.ts` : "everything goes through the small functions below so the provider can be swapped".

5. Postgres as the one piece of infrastructure

Rate limits, audit events, platform settings, notification logs, push subscriptions and cached conversions all live in the database. There is no separate Redis, queue or search server to run, secure or pay for.

> **WRITTEN RATIONALE**
> `rateLimit.ts` : Postgres is "already provisioned … no Redis, no new service, no new env vars".

6. Idempotent actions: doing it twice does no harm

The same patterns repeat: create-then-ignore-duplicate (a unique rule in the database), "update only if still in this state" (a status guard), unique event keys, and optimistic version numbers. Submitting an exam twice, tapping Pay twice, or a webhook arriving twice cannot produce two results.

> **WRITTEN RATIONALE**
> Comments throughout the exam, certificate, payment and arena code explain that a double-tap race "burns attempts" or that two simultaneous sends "cannot spend the same credits twice".

7. Never let a side effect break the main action

Sending an email, awarding a badge or writing an audit row is wrapped so that its failure never undoes the action the person actually took. Audit writing, push, analytics and AI summaries all follow this rule.

8. Privacy by design

AI assistants receive aggregated numbers, not names or question text. The service worker never caches anything personal. Real-time messages carry no answers. Notification error messages are stripped of email addresses and phone numbers before being stored. Public pages select only the fields they need.

9. Honest about what is untested

Comments repeatedly say “built to the provider’s documentation, not verified end to end” for Paystack, WhatsApp, Google Translate and similar integrations. This guide repeats those caveats rather than hiding them.

10. A paper trail for why

Comments cite the audit finding or milestone that caused each safeguard (for example “M9 audit finding #4”). That makes the reason for a rule discoverable, which is the foundation of this guide.

> **TIP**
> When you see the three labels in later chapters, the “Written rationale” boxes come from exactly this paper trail.

3 The core platform stack

The foundation tools, one by one: what they do here, why they fit, and what the trade-off is.

Next.js 14 and React 18: one codebase for pages and server

Next.js with the App Router renders pages on the server and also hosts every API endpoint (482 of them) in the same project. The platform has 220 pages; 377 files are client components for interactive parts, and the rest are server components that read the session and the database directly. Most pages are dynamic (`force-dynamic`) because they depend on who is signed in; only a few public pieces are cached.

Next.js feature used	Where and why
Server components and route handlers	Data pages read the session and database directly; endpoints enforce permissions.
Middleware (`src/middleware.ts`)	Verifies the signed session cookie, redirects signed-out visitors, hides the staff sign-in page, and passes the current path to pages and endpoints.
`next/font`	Self-hosts Fraunces and Manrope at build time: no outside font request, no layout jump.
`next/og`	Creates share-card images (1200×630 and 1080×1920) for the growth funnel.
`unstable_cache` with tags	Short caches for the public feed (60 s), landing data (300 s), guide config (about 60 s); signed-in views are never cached.
`rewrites` and `headers`	Short `/@handle` profile addresses; the configurable staff sign-in address; service-worker and public-file caching; `noindex` on staff areas.
`optimizePackageImports`	Only the icons and chart parts a page uses are sent to the phone.

NO WRITTEN RATIONALE
The repository does not record why Next.js 14 or React 18 were chosen over other frameworks.

ENGINEERING INFERENCE
One TypeScript codebase serves the pages, the API and server rendering on one Vercel deployment. For a small team this removes a separate backend service, shares types between screens and endpoints, and gives fast first loads on phones through server rendering.

TypeScript in strict mode

`tsconfig.json` sets `strict: true`. The deployment guide asks every deployer to run the type check. Combined with zod validation at every endpoint, data is checked at the edge and typed inside.

ENGINEERING INFERENCE

Types catch whole classes of mistakes (a missing field, a wrong role name) before release, which matters when 482 endpoints share the same data model.

Tailwind CSS with CSS-variable design tokens, dark mode and two fonts

Brand colours are CSS variables (`--brand-teal`, `--brand-ink`, …) mapped into Tailwind as `rgb(var(--brand-x) / <alpha-value>)`. Dark mode is the default, switched by a first-party `theme` cookie read by a tiny inline script before the first paint. A test (`designTokens.test.ts`) fails if any text and background pair drops below the 4.5:1 contrast ratio in either theme.

WRITTEN RATIONALE

`tailwind.config.ts`: dark mode "doesn't require touching the 369 existing bg-brand-* … usages". `layout.tsx`: a raw inline script rather than a React effect "to avoid a flash of wrong theme". The config describes Fraunces as a warm serif that reads as "credential" and "crafted" (used for headings and certificates) and Manrope as a clean, friendly sans that stays functional in dense screens.

PostgreSQL 16 with Prisma 5

All data lives in PostgreSQL, accessed through Prisma. The schema has 178 models, 69 enumerations and 222 indexes; about 47 columns hold flexible JSON where the shape legitimately varies (certificate layouts, audit before/after values, game item content). The 109 migrations are applied with `prisma migrate deploy`.

Capability	How it is used
Typed queries	The Prisma client gives typed, injection-safe queries across the codebase.
Migrations	Additive, reviewable steps; production builds run `migrate deploy` and fail the build if a migration fails, so the previous release keeps serving.
pgvector	`Question.embedding vector(512)` supports duplicate-question detection.
pg_trgm	Trigram indexes for fuzzy search on names, titles and handles, created by a migration "built never to fail or block a deploy".
Unique constraints and transactions	The foundation of the "do it twice, get one result" guarantees (certificates, attempts, payments, ladder points).
Advisory locks	The credit wallet takes a database lock so two simultaneous sends cannot double-spend.

WRITTEN RATIONALE

`schema.prisma` and `docker-compose.yml`: pgvector is used for duplicate detection (the Prisma client has no native vector type, so raw queries are used); the local database image is `pgvector/pgvector:pg16` because plain Postgres lacks the extension. `DEPLOYMENT.md`: use `migrate deploy`, not `db push`, because the latter "skips the migration history (and the data steps inside some migrations)".

NO WRITTEN RATIONALE

No document compares Prisma with other data layers.

ENGINEERING INFERENCE

Prisma's generated types and migration history suit a schema that changes every week; Postgres' constraints and transactions give the platform its safety guarantees without extra systems.

Neon and Vercel: managed, low-operations hosting

Production runs on Vercel with Neon as the managed Postgres. `scripts/vercel-build.mjs` generates the Prisma client, runs `prisma migrate deploy` only for production builds, then builds the app. It rewrites Neon's pooled address to the direct address for migrations, retries three times five seconds apart, and never touches the database for previews or local builds. Five scheduled jobs are declared in `vercel.json` and are protected by a secret.

WRITTEN RATIONALE

`vercel-build.mjs`: the pooled host sits behind pgbouncer, which cannot hold the lock `migrate deploy` takes (error P1002); "A sleeping Neon database can miss the first migration lock", hence the retries; a failed migration fails the build "so previous deployment keeps serving". `DEPLOYMENT.md`: Neon supports the two extensions the app needs (`vector`, `pg_trgm`). `performance.md`: put the Vercel function region in the same region as Neon.

ENGINEERING INFERENCE

Managed hosting means no servers to patch and automatic HTTPS and scaling, which fits a team without a dedicated operations engineer. The trade-off is cold starts on a sleeping free-tier database, which the performance guide documents.

Vercel Blob: direct-to-storage uploads

Pictures, resumes, flyers, certificate backgrounds, ads and videos are stored in Vercel Blob. Browsers upload straight to Blob with a short-lived token, so the file never passes through a server function.

WRITTEN RATIONALE

`materials/upload-token/route.ts`: this is "what actually fixes the 4.5MB platform ceiling". Presigned tokens are used instead of the static-token helper; database work is deliberately not done in the upload-complete callback because that callback never reaches a local machine.

Dependencies at a glance (production)

Package	Role
next, react, react-dom	Application framework and UI library.
@prisma/client (prisma for tooling)	Database access and migrations.
@anthropic-ai/sdk	Claude calls for grading, extraction, assistants and review.
ably	Server-side publishing of “something changed” signals.
jose, bcryptjs	Signed session tokens; password hashing.
zod	Validation of every request and of AI output.
resend, web-push	Email; browser push notifications.
@vercel/blob, sharp	File storage; picture resizing.
@react-pdf/renderer, mammoth, word-extractor, unpdf, jszip, node-html-parser, qrcode	PDF creation; Word and PDF reading; Office file reading; QR codes.
fabric	The certificate design canvas.
recharts, lucide-react	Charts and icons.

LIMITATION

Of the 23 production dependencies, none is a large framework beyond Next.js and React. The small number is itself a design decision: fewer moving parts to secure and upgrade.

4 Identity, security and the permission engine

How people sign in, how the platform decides what each person may do, and the safeguards around every request.

Sessions: signed tokens in a protected cookie

After sign-in the platform issues a signed token (HS256, library `jose`) in a cookie that is `httpOnly`, `SameSite=lax` and `secure` in production. The signing secret must be at least 32 characters and not the example value, or the app refuses to start. Session length depends on the role and can be changed by an administrator: staff 12 hours, trainees 7 days, employers and investors 24 hours. If those settings cannot be read, built-in values are used so a database hiccup never locks anyone out.

> **WRITTEN RATIONALE**
> `session.ts` : forcing a trainee on a multi-week course to sign in every 12 hours is “real friction with no corresponding security benefit”; a 24-hour employer session is “long enough to not be annoying … short enough to matter for an account holding real applicant contact information”. Cookie lifetime and token expiry now come from one number because an earlier hand-kept second value would have signed a new role out early.

> **ENGINEERING INFERENCE**
> `jose` runs in the edge middleware runtime (where the common `jsonwebtoken` package does not), so the same library verifies the session in middleware and on the server.

`getSession()` re-checks the account on every request: a suspended staff member or a removed organisation teammate is signed out on their next click, not when the token would have expired.

Passwords and sign-in routes

Passwords are hashed with `bcryptjs` (cost 10). Each account type has its own sign-in and reset routes (trainee, staff, employer, investor, training organisation). Trainees may also use Google sign-in. Training organisations, owners and teammates alike, can reset or change their password themselves. Reset links last one hour and work once.

> **ENGINEERING INFERENCE**
> bcryptjs is pure JavaScript, so it needs no native build step on a serverless host. Its cost factor of 10 is a common default; no written rationale was found for the number.

Middleware: a fast first check, never the only check

The middleware verifies that a valid session cookie exists and redirects signed-out visitors to the right sign-in page, remembering where they were going. It deliberately does not decide roles: every

endpoint and page still calls the permission check itself. Middleware also hides the internal staff sign-in page behind a configurable address and marks staff areas `noindex`.

> **WRITTEN RATIONALE**
> `middleware.ts`: middleware only answers "is there a valid session"; "API routes still call requireRole(...) themselves as the real enforcement point". `staffAccessPath.js`: "Hiding the address is not the protection: every staff page and API still checks the session and role on the server."

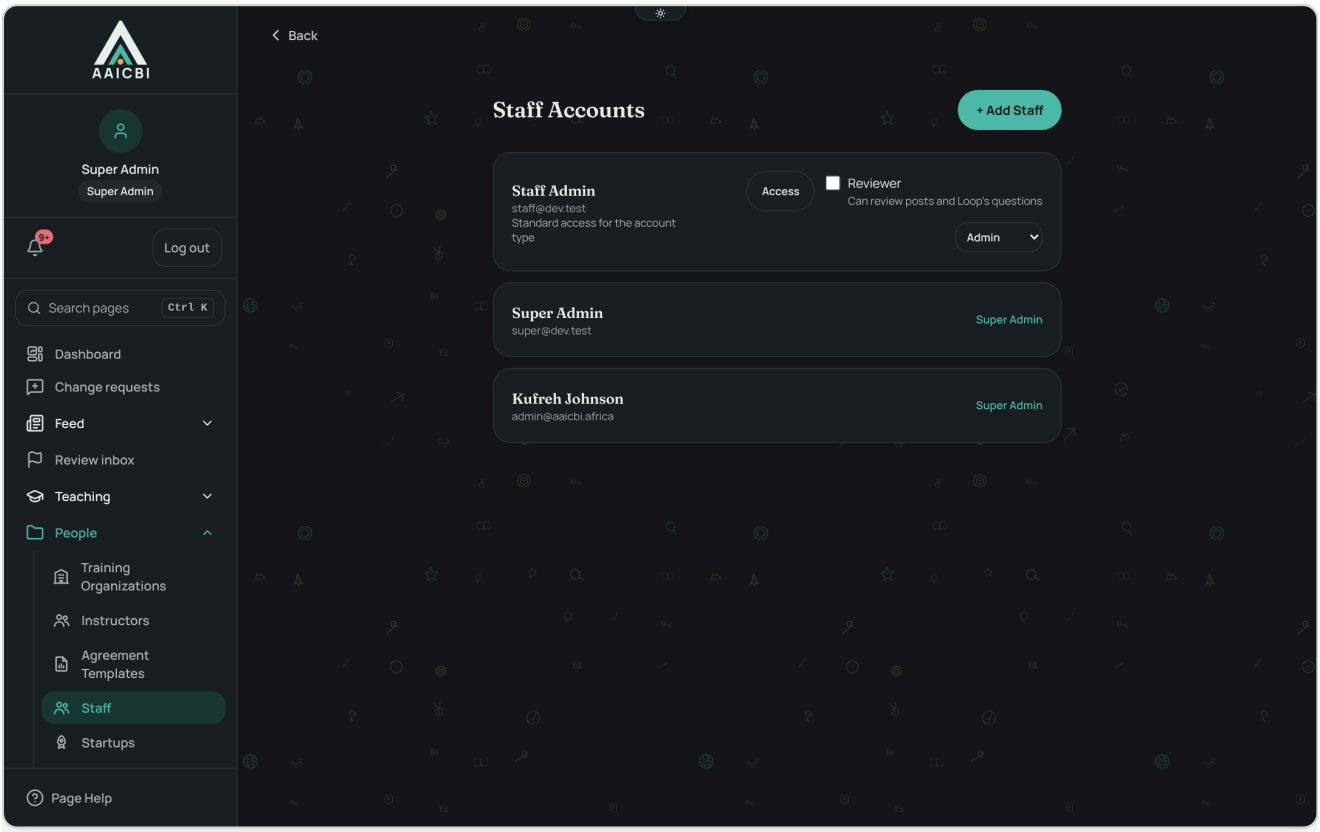
The permission engine (RBAC)

Before this engine, access was decided in several places at once: the role, a navigation table, per-person studio keys, a reviewer flag, the organisation team role and about 398 separate role checks. The engine replaces that with one resolver.

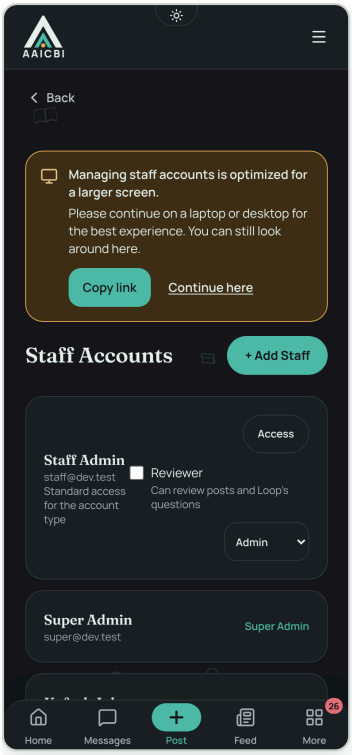
Part	What it does
Catalogue (`rbac/catalogue.ts`)	Every permission is named `category.feature:action` — for example `gaming_studio.games:publish` or `gaming_ops.results:export`. Actions are view, create, edit, delete, manage, approve, publish, moderate, export, configure and assign_permissions. About 90 features across 14 staff categories plus an organisation-only category. A subset is marked high risk.
Role templates (`templates.ts`)	Eight ready-made roles as data, for example Gaming Studio Administrator, Gaming Operations Moderator, Platform Administrator, Reports Officer. Custom roles can be saved.
Resolver (`resolve.ts`, pure)	Super Admin: all staff permissions and cannot be locked out. Inactive or non-staff: nothing. Staff with no role set: exactly the legacy access they already had. Otherwise: template or custom role, plus personal grants, minus personal denials, then gates (no Studio permission counts without Studio access).
Guard (`guard.ts`)	Built on the older role check so sign-in, team status and billing pauses still apply first; reads the database each request; takes the organisation from the session; answers "not found" and records an audit event for cross-organisation attempts.
Anti-escalation (`authority.ts`)	Nobody edits their own permissions; Super Admin accounts cannot be edited in the app; a non-super manager needs the assign-permissions permission, may only hand out keys they hold themselves, and can never hand out high-risk keys.
Audit (`audit.ts`)	One `AuditEvent` table: actor, role, action, resource, organisation, outcome (OK, DENIED, FAILED), reason and before/after values. Writing is best effort and never records passwords, tokens or message text.

> **WRITTEN RATIONALE**
> `docs/rbac-audit-and-plan.md`: role ADMIN meant two different things (an AAICBI admin and a training organisation signed in as an admin), and "this is how organizations reached the Game Studio". Existing access is preserved by legacy defaults so "nobody gains or loses access on deploy"; the migration is additive; and 398 existing checks "cannot be rewritten safely in one change", so categories move over one at a time.

Training organisations have their own small permission set, `org_games.*`, resolved from their team role (Owner and Manager may moderate; Editor and Viewer may only view) and from a per-organisation policy that the Super Admin controls with nine switches. Organisation team roles are enforced on every request: Owner everything; Manager everything except billing; Editor builds content but cannot publish, delete, change the public profile, billing or team; Viewer reads only.



COMPUTER



PHONE

Figure 2. The staff page, where a Super Admin gives each staff member a role template or individual permissions.

Rate limiting, request safety and validation

Safeguard	How it works	Why it is built this way
Rate limiter (`rateLimit.ts`)	Fixed-window counters in a Postgres table, about 79 call sites. Examples: sign-in 5 per 15 minutes, registration 10 per hour, AI chat 30–40 per 15 minutes.	Written: an in-memory limiter on serverless “could be far weaker than it looks”; Postgres is already there. The small race between read and write is openly acknowledged.
SSRF guard (`ssrfGuard.ts`)	Before the server fetches a staff-provided web address, it resolves the name and refuses private, loopback, link-local and cloud-metadata addresses; it fails closed.	Written (M40 audit): checking only the protocol is not enough. The disclosed limit: it does not defend against DNS rebinding.
Validation (zod)	About 282 files validate input with zod; AI replies are validated too.	One declared shape per endpoint keeps invalid data out of the database.
`withApiErrors`	A wrapper used by 439 route files that turns errors into clean JSON (409 for blocked deletions, 404 for stale sessions, logs only real failures).	Written: keeps route files free of repeated try/catch code.

Safeguard	How it works	Why it is built this way
Webhook and cron secrets	The Paystack webhook checks an HMAC-SHA-512 signature on the raw body; cron endpoints require a bearer secret and give an identical 401 whether the secret is missing or wrong.	Written: do not help an attacker work out which part failed.
Cookie consent	No visitor-tracking cookie or event is created until a visitor accepts.	Written in `CookieConsentBanner.tsx`.

IMPORTANT

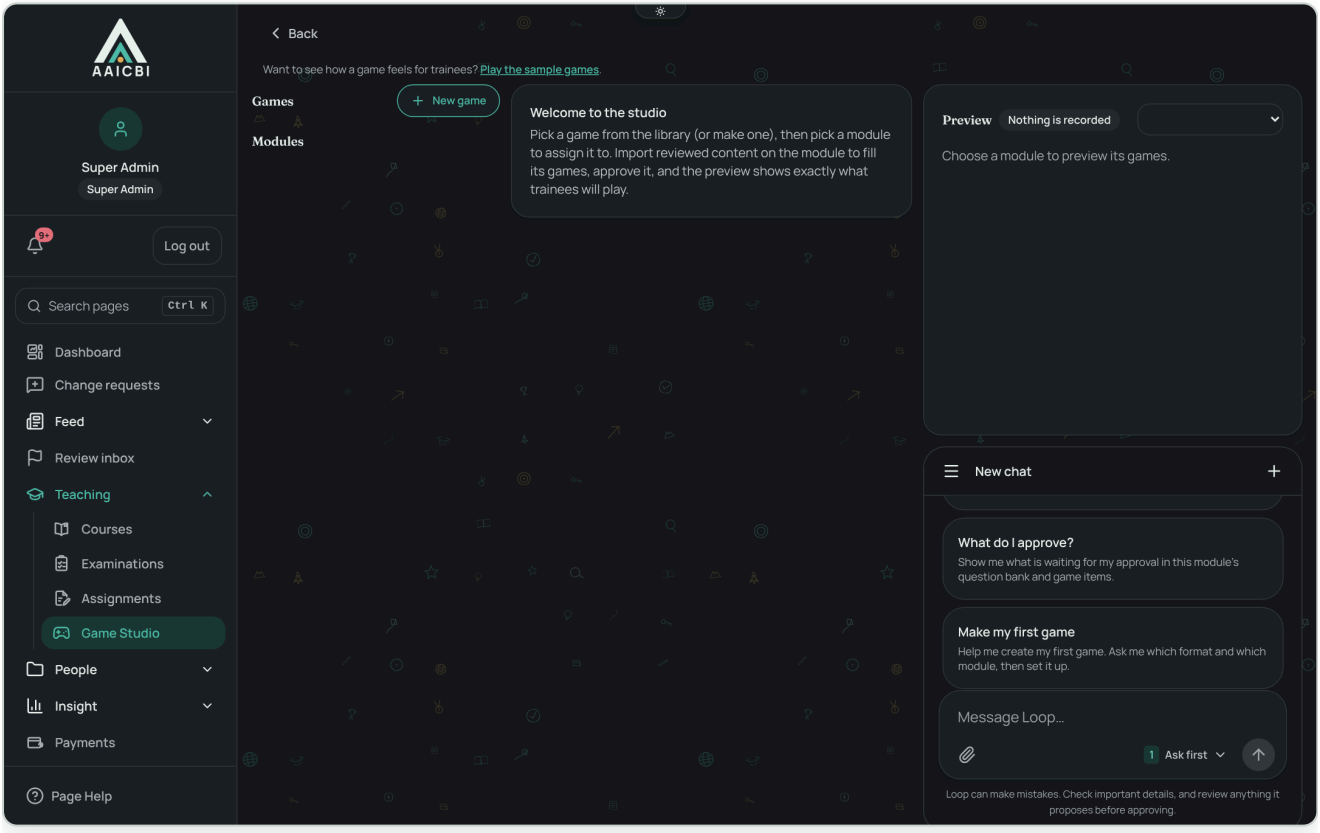
What the code does **not** contain: no Content-Security-Policy, HSTS or X-Frame-Options headers are set by the application, and there is no CSRF token mechanism. Protection relies on the `SameSite=lax` httpOnly cookie, JSON-only endpoints and server-side role checks, and HTTPS is provided by the host. No document explains these choices. They are listed again in the final chapter as items to review.

5 The Game Studio

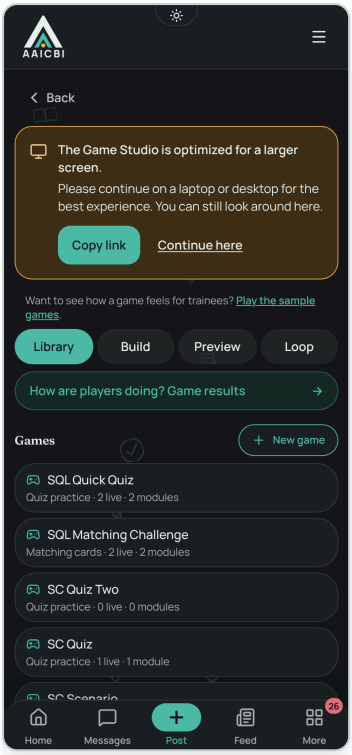
The tool where learning games are built, reviewed, tested and switched on, and the engine that serves and grades them.

What the Game Studio is

The Game Studio is one page, `/admin/studio`, where games are made, played exactly as trainees will play them, and assigned to course modules. It replaces moving between the assessment page, the question bank and the practice settings. It has four panels: **Library** (games and every course's modules), **Build** (the selected game's items, quiz pools, practice rules, question bank, tidy tools and import), **Preview** (plays the module's game as a trainee would, with drafts marked and nothing recorded) and **Loop** (a compact AI assistant that knows which module and game are selected). Below a wide screen the four panels become tabs.



COMPUTER



PHONE

Figure 3. The Game Studio. Only the Super Admin and staff who were granted Studio access can open it; training organisations never can.

Who can use it

Studio access is a permission, `gaming_studio.access:view`, and each action then needs its own permission: create, edit, delete, publish, manage (attach to a module), preview, manage assets, configure settings, configure the engine, manage Loop, manage the question bank and approve bank questions. Viewing results and moderating games belongs to a different family, `gaming_ops.*`, and **never opens the Studio**. Training organisations get a separate, limited Games section for their own courses.

> **WRITTEN RATIONALE**
> `docs/rbac-audit-and-plan.md` : the Studio was reachable by organisations only because role ADMIN meant two things; the fix is separate permission families so building games and watching games are different jobs. `rbac/legacy.ts` : “nobody but a Super Admin has any gaming_studio.* permission until it is granted.”

The four game formats

Format	What the learner does	Rules enforced when it is saved
Quiz	Picks the correct option; questions come from the module’s approved practice pool with hints, XP and mastery tracking.	Bank rules: exactly one correct answer, at least two options, objective-tagged, human-approved.
Matching	Pairs terms with meanings.	3 to 6 pairs; lefts unique and rights unique (ignoring case); text up to 300 characters.

Format	What the learner does	Rules enforced when it is saved
Ordering	Puts steps in the right order by dragging.	3 to 8 unique steps stored in the correct order.
Scenario	Chooses the best action in a realistic situation and sees the outcome of each option.	2 to 4 options, exactly one best, each with text and an outcome.

A game is built once and can be assigned to many modules. Going live is gated: a quiz game needs every learning objective to have enough approved questions (10 for practice and 6 for the formal assessment) and only one quiz game can be live per module; item games need at least one approved item. Deleting the last approved item pauses live games automatically.

> **WRITTEN RATIONALE**
> `quest/studio.ts` and `docs/game-studio.md`: the existing Knowledge Quest settings stay "the source of truth" for quiz practice, so switching a quiz game live or off simply drives the existing practice setting.

From idea to live game: the lifecycle

Stage	What happens	Who or what is the gate
1. Content arrives	Reviewed content is imported from a file (see below), or a person writes items by hand.	Import never calls AI and never publishes.
2. Draft	Imported items become drafts; imported quiz questions are marked "passed review" but still wait for approval. Trainees never see drafts.	Automatic.
3. Review and edit	A person edits an item; editing returns an approved item to draft because "edited content must be looked at again".	Human.
4. Approve	Approve items and bank questions one by one or in bulk. A flagged item can never be bulk-approved.	Permission `bank:approve`; Loop's approval action is always shown for confirmation.
5. Preview	Play the game as a trainee would, including drafts, without recording anything. Graded by the same grading function.	`testing:view`.
6. Go live	Attach to a module and switch on; optionally "show on the feed" for signed-in visitors.	`games:manage`; `games:publish` only when the live state actually changes; logged as an audit event.

The Question Bank inside the Studio

The bank keeps two sides apart: **Games and practice** (10 questions per learning objective) and **Assessment** (6 per objective), so formal-exam questions are never shown in practice. Questions pass

through states: awaiting review, ready, flagged, approved, rejected. Studio actions are approve, assign to a side, copy to the other side, reject and edit. Every change writes a `QuestionBankEvent` so there is a history of who did what.

Importing reviewed content, and tidy tools

- **Import** takes a file shaped as `{course, module, objectives:[{quiz, matching, ordering, scenario}]}` (up to 20 objectives and 40 items per format). It always previews first; each objective in the file is matched to an existing one by word similarity, or marked new or skipped. It reserves a share of quiz questions (default 6 per objective) for the assessment side, creates the four games switched **off**, and is repeatable: a signature check means importing the same file twice adds nothing.
- **Tidy tools** merge duplicate objectives, rename them (409 if the wording already exists), delete empty ones, and move spare practice questions to the assessment side until 6 per objective remain. None of these use AI.

> **WRITTEN RATIONALE**
> `quest/contentImport.ts` and `docs/game-studio.md`: nothing becomes playable until a person approves both bank tabs, approves drafts and switches games on; the importer "never calls Anthropic".

How the content is written: the module-content workflow

Game and practice content is produced by a documented workflow (`.claude/skills/module-content`). Lesson files are turned into learning objectives, which a person confirms. Writer agents (Claude Sonnet) then write 20 quiz questions, 20 scenarios and 10 to 20 matching and ordering items per objective, using only the lesson text. Reviewer agents (Claude Opus, a **different** model) answer every item blind — they never see the intended answer — and flag anything unsupported, ambiguous, off-objective, duplicated, a giveaway, unclear or factually wrong. Writers fix the flags, and the cycle repeats up to three times. Disputes about a quiz or scenario answer go to a separate Opus tie-break rather than back to the writer.

Result from the first module (recorded in the workflow notes)	Value
Items flagged after review pass 1, 2, 3	261 of 640, then 125, then 59
Disputed answers sent to tie-break	35, and the writer's key was right in all 35
Commonest defect	The "giveaway": the correct option is longer than the others
Hardest format	Ordering — when the source does not fix a sequence, delete rather than rewrite

WRITTEN RATIONALE

`bankModels.ts`: "One model WRITES the questions and a DIFFERENT model REVIEWS them, so a question is never checked by the model that wrote it (the same model tends to repeat its own mistakes)." The workflow notes add: "Never trust a lone disagreement: tie-break first."

Loop inside the Studio: an assistant that proposes, never decides

Loop is the platform's Claude-powered assistant, shown in compact form in the Studio. Claude never writes to the database itself: it names an action, the server validates it with a schema and writes a plain-words summary, and the chat's autonomy level decides whether it runs now or shows an approval card. Every proposed or executed action is stored with its exact input, and approving runs exactly that recorded input.

Autonomy level	Behaviour
1. Ask first (default)	Every action shows an approval card.
2. Balanced	Low-risk actions run; high-risk actions ask.
3. Autonomous	Actions run, except those marked always-confirm.

- Always-confirm actions include approving game items and approving bank questions, so Loop can never approve its own drafts. Messages to more than 25 people always ask first.
- Studio actions are `create_game`, `set_game_rewards`, `assign_game` (high risk only when it makes a game live), `approve_game_items` and `approve_bank_questions`.
- Permissions are enforced inside the tool loop, so a request for something outside the person's permissions fails even if the prompt never offered the tool. Staff without full access get a limited prompt that lists only what they were granted and start at level 1 unless a Super Admin raised their level.
- The model is `claude-sonnet-4-5` by default (override `LOOP_MODEL`); replies stream; up to 12 tool rounds per message.

WRITTEN RATIONALE

`docs/loop-command-center.md`: "Claude never writes anything itself. It names an action; the server validates it, writes a plain-words summary, and the chat's level decides whether it runs now or shows an approval card." "Loop can do exactly what its tools and actions allow, no more." "Approving runs exactly the recorded input."

ENGINEERING INFERENCE

Forcing the model through validated actions limits the damage of a mistaken or manipulated answer to a proposal a person can read, which is the safest place to put an AI in a workflow that affects learners.

The engine: serving and grading, with cheating in mind

Every number of the learning design lives in one pure file, `quest/core.ts`, which has its own tests. The database half (`server.ts`, `studio.ts`) serves and records.

Rule	How it works	What it prevents
Answer key stays on the server	Matching serves the left items in order and the right items shuffled with no mapping; ordering serves shuffled steps (re-shuffled up to 5 times so they are never served already in order); scenarios serve option texts without the "correct" flag or outcome. The key and outcomes return only after the answer.	Reading the answers from the page or network.
One question at a time	The browser asks for each item separately; there is no batch download.	Pulling every question in advance.
Server clock	The play row is created when the item is served; time taken is now minus that time. A resumed item is re-served, not re-rolled, for 10 minutes.	Claiming a fake answer time.
Double-submit guard	An answer is claimed with a conditional update (only if not yet answered).	Double XP from a double tap.
Too-fast flag	Minimum seconds = the larger of 3 or the word count divided by 4, rounded up. A faster correct answer earns no XP and is not mastery evidence; a faster wrong answer still counts against accuracy.	Guessing at speed to farm XP or fake mastery. There is no warning or lock-out; the answer simply earns no credit.
Tamper checks	Ordering must be the same length, unique and made of the real steps; matching must be complete; a scenario choice must equal an option.	Crafted submissions.
Partial credit	Matching and ordering score the share of pairs or positions right; scenarios score 1 or 0. Only full marks count toward mastery.	Rewarding half-knowledge as mastery while still paying small XP.

Mastery, XP and the practice design

Rule	Value
Mastery of an objective	At least 6 different questions correct, accuracy of at least 80% over the last 10 answers, and evidence on at least 2 separate days.
Mastery stays	Until 30 days of inactivity or window accuracy falls below 60%, then "needs refresh".

Rule	Value
XP	Correct 10; with a hint 5; correct on a same-day retry 5; bonus +5 once for a question missed on an earlier day; streak day +10; objective mastered +50; module completed +100; daily cap of 100 for answer XP.
XP ledger	Append-only with a unique key per award, so races and rule changes never take earned XP away. XP never counts toward certificates.
Day boundary	Nigerian time (UTC+1, no daylight saving) so a "day" matches the learners' day.
Question choice	Weakest objective first; within it never seen, then last wrong, then least recently answered; never the previous question again.
Rounds	Rounds of 10, requested one question at a time.
Formal exam link	Critical objectives must be cleared as well as the pass mark; the exam draw guarantees questions for each critical objective.

WRITTEN RATIONALE

`docs/knowledge-quest-design.md` : "A trainee progresses because they show competence, not because they spend time or collect points." Formal result, mastery and XP are kept apart. Practice answers are stored raw, so thresholds can change "with no data migration". The numbers are "starting values, to be tuned with real trainee data".

NO WRITTEN RATIONALE

The specific numbers (80%, 6, 10, 2 days) have no written justification beyond "first guesses; low stakes".

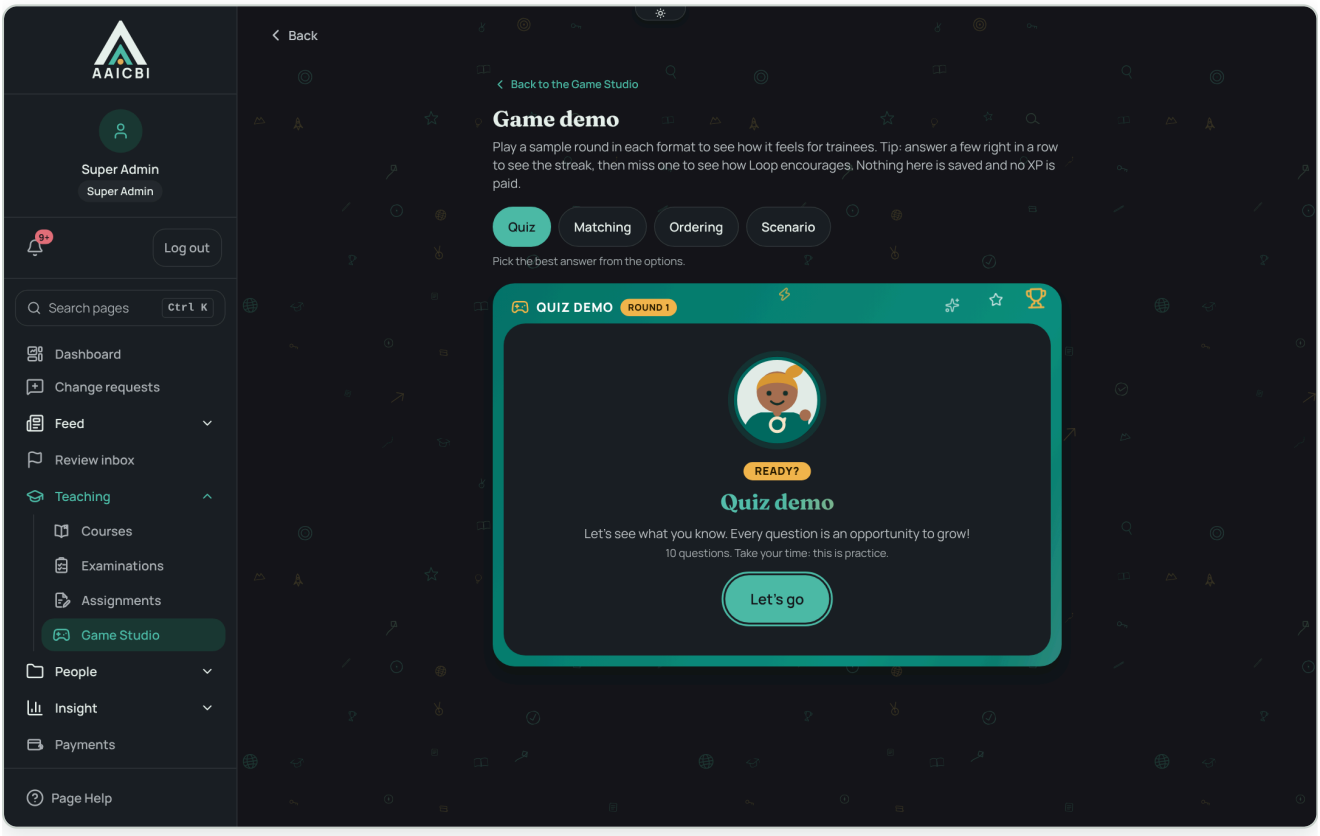
ENGINEERING INFERENCE

Requiring different questions on separate days makes it hard to pass by memorising one question or cramming in one sitting.

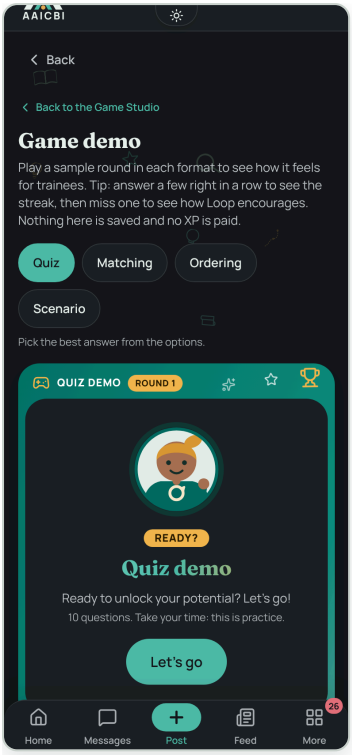
The front end of a game

Technique	What it is	Why it fits
CSS-only animation	All game motion uses CSS keyframes on transform and opacity (the `gx-` classes). A small confetti is drawn from page elements that remove themselves. No animation library is installed. A global reduced-motion rule switches it all off.	Written: "No animation libraries were added." Inference: transform and opacity animations stay on the graphics processor, which matters on low-end Android phones.

Technique	What it is	Why it fits
Web Audio synthesis	Sounds are tones made in the browser (select, correct, wrong, streak, completion) — no audio files. Sound is off by default; the preference is remembered; audio starts only after a tap, as browsers require; a miss sounds “soft and falling, never a buzzer”.	Written: “no audio files to download, license or lose”.
Drag-and-drop ordering	Pointer Events with pointer capture; only the handle drags so scrolling still works on touch; the row follows the pointer; the page scrolls near an edge; the Up/Down arrow keys work on the handle.	One implementation shared by practice and the arena; keyboard users are not locked out.
Accessible by design	Everything works by keyboard; feedback and the countdown are announced politely; progress pips carry text, not only colour.	Written in `docs/game-experience.md`.



COMPUTER

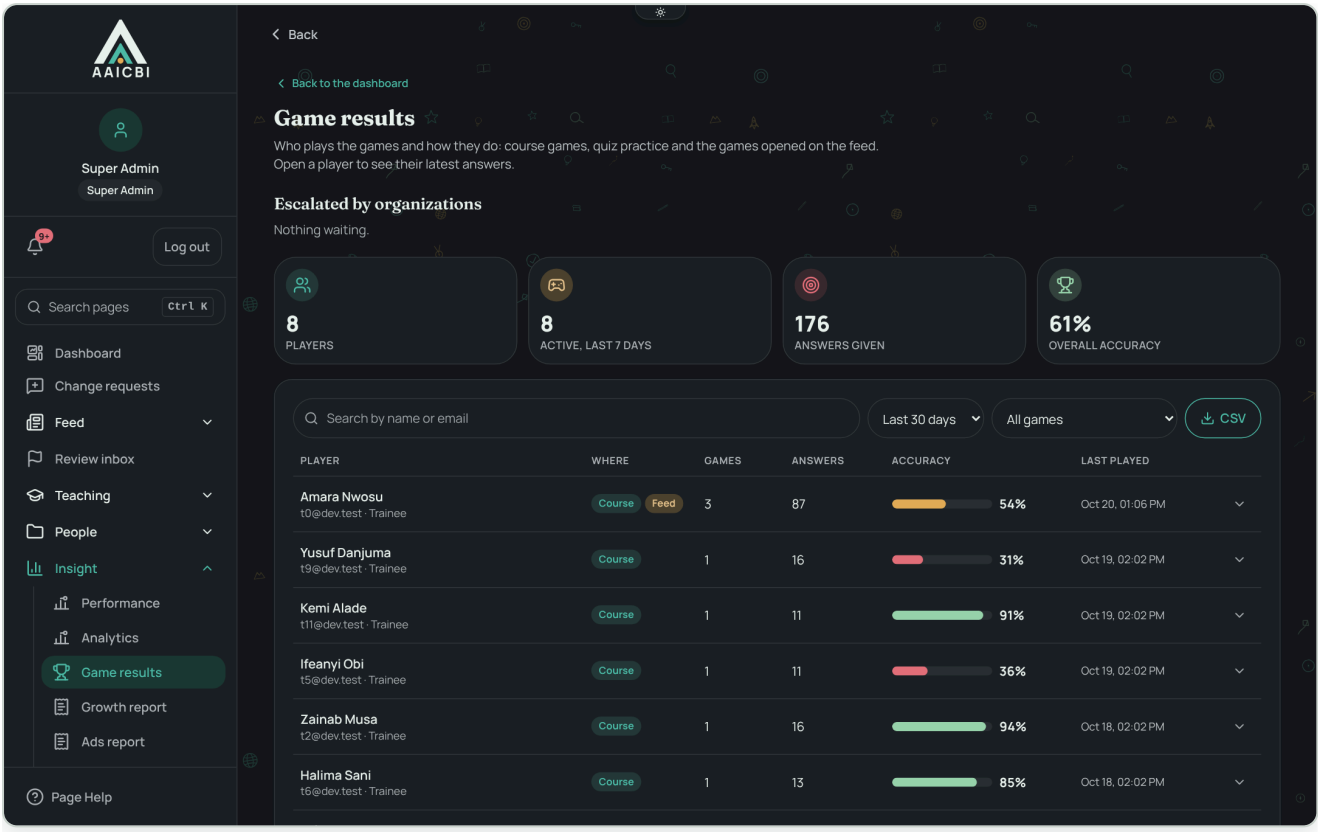


PHONE

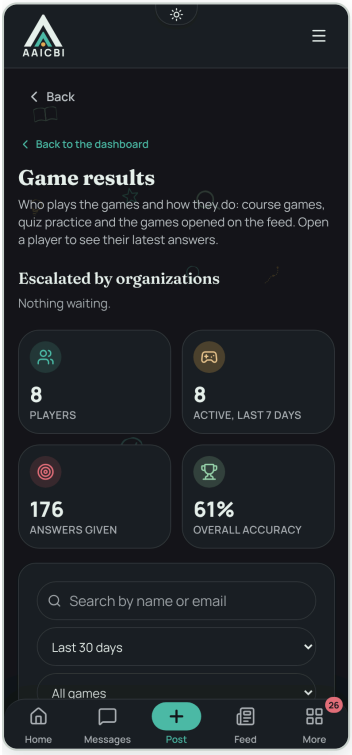
Figure 4. The sample-games page: ten sample items per format, graded by the same functions the server uses, nothing saved.

Settings and results

Global arena switches and rules live at `/admin/arena` and need `gaming_studio.settings:configure`. Results live in a separate page that merges course games, quiz practice and games opened from the feed; viewing needs `gaming_ops.results:view` and download needs export permission and is audited.



COMPUTER



PHONE

Figure 5. Game results: every player with answers, accuracy and latest answers, kept separate from the Studio.

Why this design is a strong fit for AAICBI

- **Teachers stay in charge.** AI helps produce and review content, but a person approves every question and item, and the assistant cannot approve its own work.

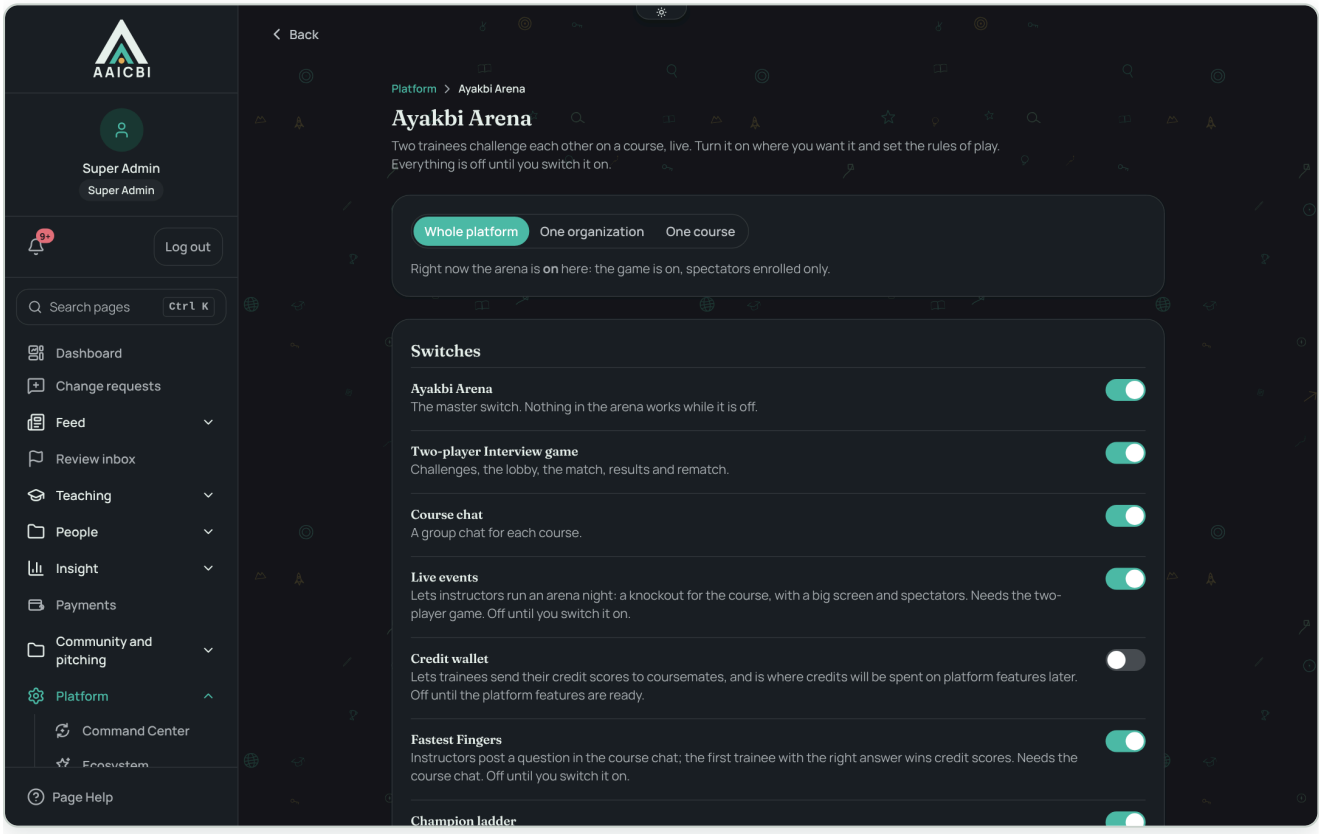
- **Fair for learners.** The answer key never leaves the server, timing is server-side, and mastery is based on different questions on different days rather than points.
- **Low operating cost.** Imports, tidy tools and the bank call no AI at run time. Content is produced once, reviewed, and reused by every learner.
- **Works on modest phones.** One question at a time, CSS animation, synthesised sound and no large libraries keep games light.
- **Safe to change.** Every threshold is in one tested file, and stored raw answers mean a rule change never needs a data migration.

6 The Ayakbi Arena, real time and the growth games

Two learners play each other live on their phones. This chapter explains how that stays fair, fast and cheap.

What the Arena is

The Ayakbi Arena lets two trainees on a course challenge each other to a live question game, chat in a course group chat, answer quick “Fastest Fingers” questions from an instructor, climb a ladder, watch live matches and join knockout events. It is **off everywhere until a Super Admin switches it on**, per platform, organisation or course, and the narrowest setting wins. Instructors can change the rules of play on their course but never the switches.



COMPUTER

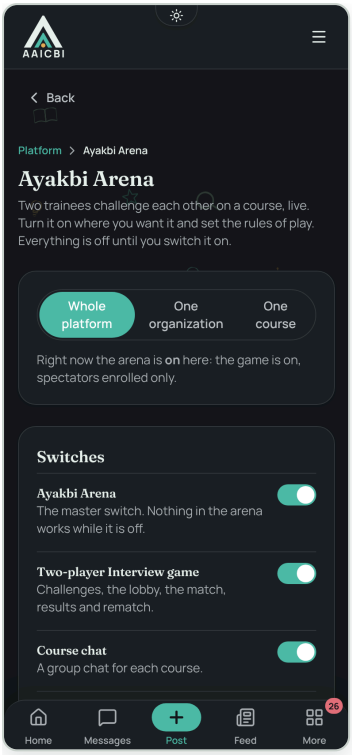


Figure 6. The Super Admin's Arena settings: switches and rules, for the whole platform, an organisation or one course.

The architecture: the database decides, real time only nudges

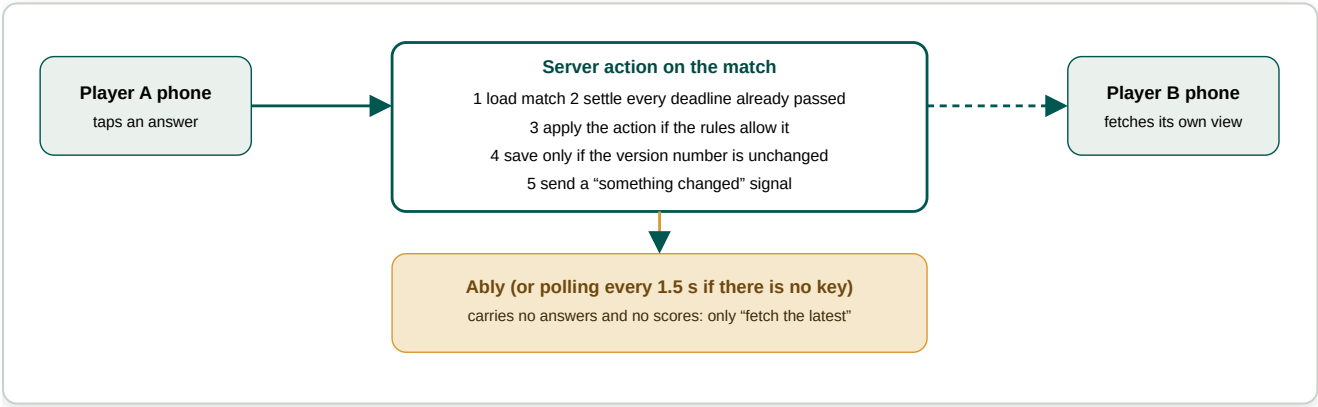


Figure 7. One move in a match. The state of the game is always in the database; real time only tells the other phone to look again.

- **Deadlines are the server's.** Every request first moves the match forward past any deadline that has already passed, so a match never depends on a browser staying open. A match can progress, time out or forfeit (three missed turns in a row) with nobody watching.
- **Versioned saves.** A save succeeds only if the match's version number is unchanged; otherwise it retries up to four times and then answers "the match changed while you were acting". Two players tapping at the same moment cannot corrupt the score.
- **Each player gets a private view.** The page is rebuilt for each player and never contains the answer key, the other player's wrong answer, or the question while the game is paused.
- **The server picks the question** behind a chosen number on the board, "so nobody can peek".

WRITTEN RATIONALE

`arenaCore.ts` and `arenaView.ts`: "any request moves a match forward past deadlines that have already passed, so the game never depends on a browser staying open"; "The browser never receives the answer key, the other player's wrong answer, or the question while the game is paused."

Ably, the Server-Sent Events stream and the polling fallback

Ably is used only on the server (`Ably.Rest`), and only to publish a tiny message with a version number. Browsers receive a short-lived, **subscribe-only** token (one hour) scoped to their own channels, and listen over a plain event stream — the Ably JavaScript library is not shipped to phones. If there is no Ably key, or the stream drops, the page polls every 1.5 seconds, pauses while the tab is hidden, backs off to every 8 seconds as a safety net when live, and renews its token at 50 minutes. A page also notices when the server is a newer release than its own and reloads once.

WRITTEN RATIONALE

`realtimeCore.ts`: browsers can only subscribe; "Every message is sent by our server after it has checked the rules, so nobody can forge a score … Messages carry no answers and no private details". `realtime.ts`: it is "the only file that knows about Ably" and "with no ABLY_API_KEY the arena still works: pages simply check for changes every second or two".

ENGINEERING INFERENCE

Using the plain event stream instead of the vendor's browser library keeps the phone download small and avoids bundling a library for older phones; a managed real-time service avoids running and scaling a WebSocket server of our own.

The rules of the match (values from the code)

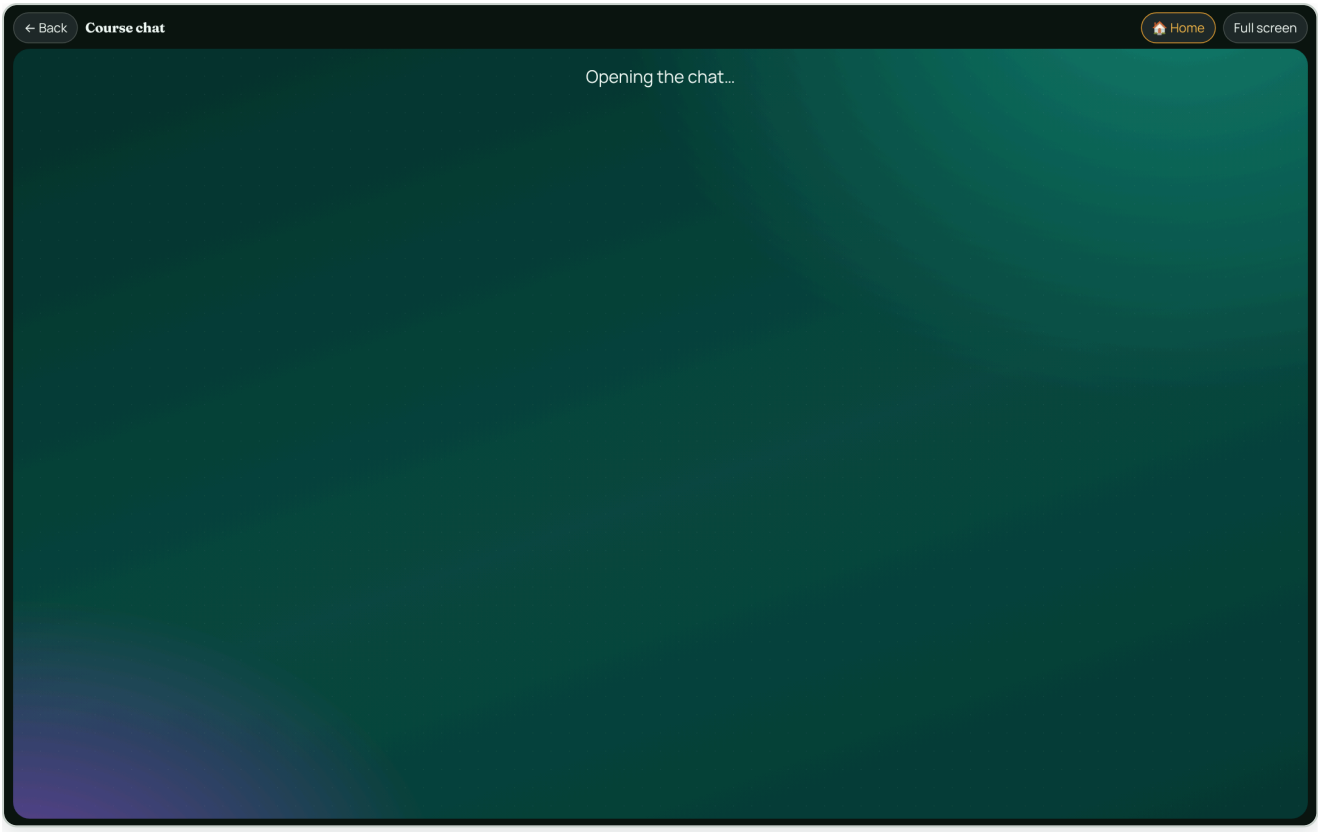
Rule	Value
Question formats	All four: quiz, matching, ordering, scenario, graded by the same functions as the Studio. Only approved practice questions and approved items are used — never the assessment pool.
Base points	Quiz 80, matching 100, ordering 120, scenario 100; a speed bonus up to 20% of base on a first correct answer.
Steal window	20 seconds for the other player to answer a missed question for base points plus the bonus (default +50%: 80 becomes 120).
Question timer	Default 60 seconds (instructor range 10 to 180); 20 questions per match by default (4 to 30).
Pauses	2 per player of up to 60 seconds, then the game resumes by itself.
Invitations	Wait 10 minutes if the opponent is online, 60 if away; a lobby lapses after 15 minutes; a match is abandoned after 30 minutes without activity.

Rule	Value
Chat	Match chat up to 140 characters; course chat up to 500 characters, last 60 messages from the last 7 days; repeat text within 10 seconds is ignored.

WRITTEN RATIONALE
The code is authoritative: `STEAL_SECONDS = 20`. The older Release 2 text in `docs/gaming-arena.md` says 5 seconds; Release 3 and the code say 20. Phones play upright because, in the document's words, "Kahoot, Trivia Crack, QuizUp and Duolingo all do: one hand, thumb reach, text-heavy questions read naturally, and nobody has to turn the phone".

The ladder, Fastest Fingers and the credit wallet

Feature	How it works	Safeguard
Elo ladder	Everyone starts at 1000; K = 32; expected score is 1 / (1 + 10^((theirs − mine) / 400)); the change is round(K × (result − expected)). Tiers: Bronze under 1000, Silver 1000, Gold 1200, Platinum 1400, Champion 1600. A player appears after 5 counted matches.	The same two players count at most 3 matches a day; a result settles once (a unique record absorbs a repeat); a moderator can void a match and ratings are restored; the weekly champion is found the first time anyone looks after the week turns, with no scheduled job.
Fastest Fingers	An instructor posts an approved or custom question with 1 to 100 credits; each trainee gets one try.	The winner is decided by a single atomic update ("only if still open and no winner"), so two simultaneous correct answers cannot both win; the key stays hidden until it closes.
Credit wallet	Credits from Fastest Fingers can be sent to a coursemate (1 to 100 at a time, up to 300 a day, never more than the balance). It has its own Super Admin switch, off by default.	A database advisory lock inside a transaction stops two simultaneous sends double-spending; every award has a unique reason and reference so repeats are ignored.
Spectators and live events	Enrolled learners can watch a live match and send emoji cheers; instructors can run a single-knockout event seeded by ladder rating, with a big-screen view.	Watchers cannot be players or suspended; viewing changes nothing in the match.
Moderation	Reports (cheating, abuse, other), ending a live match, voiding a result, removing a message, suspending a player from one hour to 90 days.	Every decision stores who did it; organisations moderate only their own courses, with a required reason.



COMPUTER



PHONE

Figure 8. The course chat on a phone, with challenge cards and reactions.

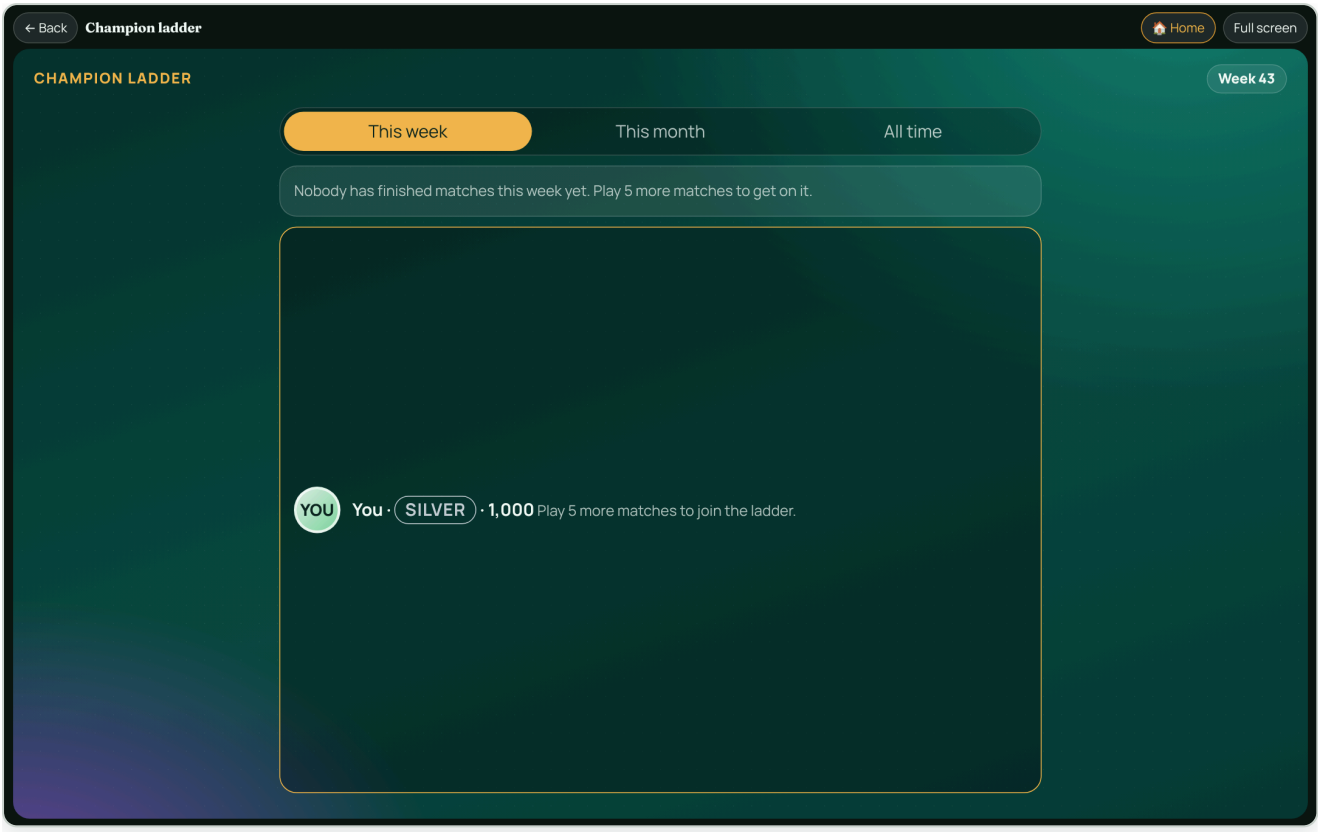


Figure 9. A course ladder on a phone: podium, tiers and the player's own place.

Notifications for games

A challenge creates a bell entry and a web push that stays on screen until tapped and vibrates the phone (`requireInteraction`, `vibrate` in the service worker). The app icon shows one combined

attention number: unread group chats, direct and cohort messages and a waiting challenge. If no one answers, the challenger is told, and a daily job plus a lazy sweep (at most once every 30 seconds per server) settles lapsed invitations.

Growth games: the public taster, duels, squads and result cards

A public link lets anyone with a phone try a five-question taster of a course with no account, then offers Module 1 free and payment after that. It draws only from the course's approved practice questions, gives 18 seconds per question, and returns the answer key only after each answer. The funnel is off by default behind four switch layers and a readiness check, and every change is audited.

- **Duels and squads.** A player can challenge a friend to the same five questions; the higher score wins and a clearly faster run (at least one second) breaks a tie. Squads of up to 50 play the captain's questions and add their scores. Names are free text from anonymous people, so administrators can rename or remove them.
- **Result cards.** The share image is drawn with `next/og` (JSX to PNG) at 1200×630 (link preview) or 1080×1920 (WhatsApp Status), cached at the edge for a day, and shows only what the player chose to share. A percentile appears only after 30 finishers, "so a small crowd never produces a made-up claim".
- **Leads and consent.** A saved score stores the phone number in +234 form, the exact consent wording and time, and a private opt-out link. Follow-up messages obey quiet hours (21:00–07:00 Nigerian time), a daily cap and stop at opt-out or payment.
- **Feed games.** A live game can be shown on the feed; any signed-in person can try a round, each graded answer is recorded, and the page says so. Answers earn no XP.

> **WRITTEN RATIONALE**
> `growth/*` and `docs/growth-funnel.md`: the assessment pool is never served; every switch is off until a Super Admin turns it on; the follow-up WhatsApp path was tested only against a stand-in for Meta's service.

Why the Arena design suits AAICBI

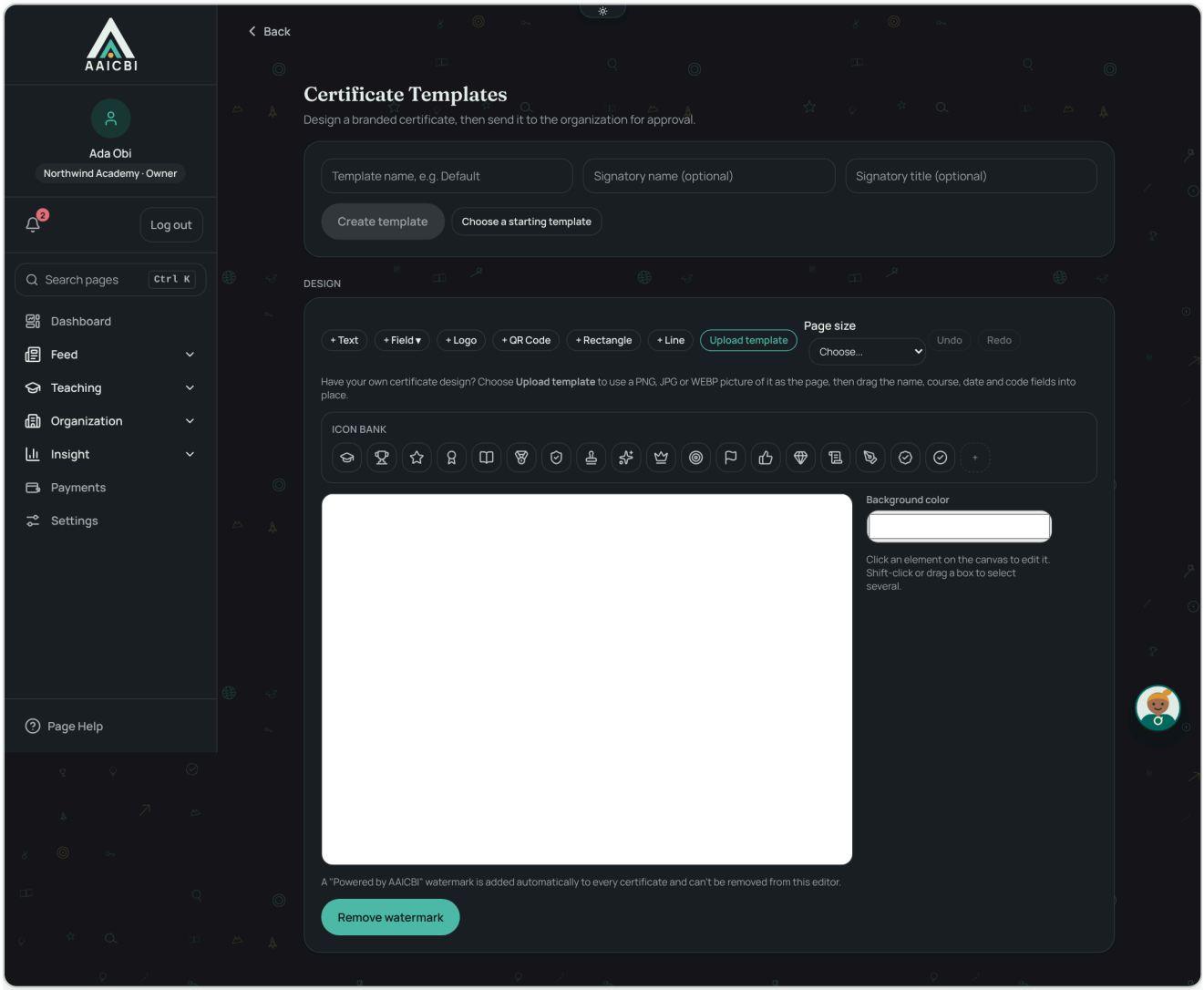
- **Fair and tamper-proof** because the server owns the clock, the questions, the scores and the order of moves.
- **Cheap to run** because there is no custom socket server; a one-line signal service plus the database does the job, and polling means it still works if the signal service is off.
- **Light on phones**: portrait layout, a plain event stream, CSS animation and synthesised sound.
- **Safe by default**: everything is off until switched on; moderation requires reasons and is audited.

7 The Certificate Studio and document engines

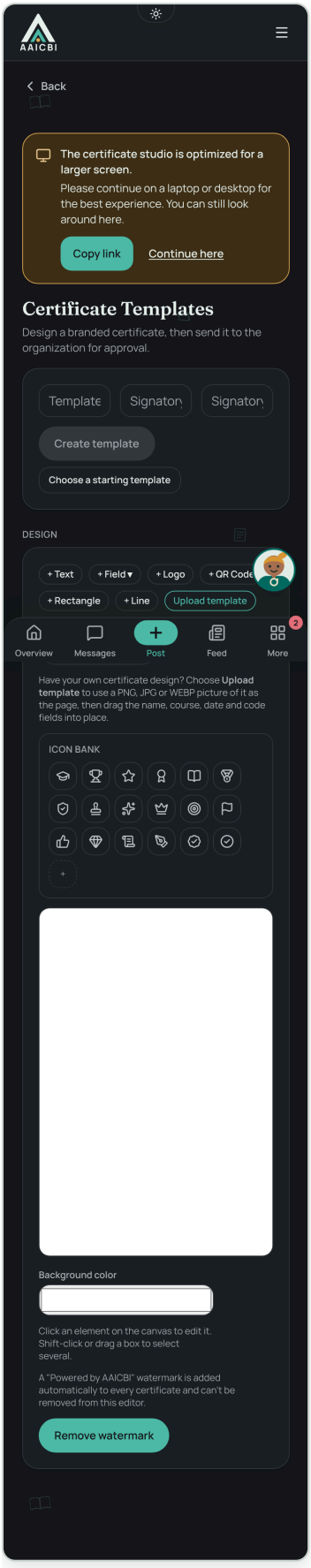
How an organisation designs a certificate, how it is issued and proved genuine, and how every other PDF and image is made.

What the Certificate Studio is

Each training organisation can design the certificate its trainees receive. The Studio is a drag-and-drop canvas where the organisation places text, dynamic fields, its logo, a QR code, shapes and icons on a page, or uploads its own background image. A new design starts from one of four presets (AAICBI Classic, Modern Minimal, Gold Elegant, Bold & Teal) or from blank. Organisations design on a larger screen; the Studio shows an “optimised for a larger screen” note on phones.



COMPUTER



PHONE

Figure 10. The Certificate Studio: create a template, choose a starting design, place elements on the canvas, upload a background.

The editor: Fabric.js in the browser

The editor uses **Fabric.js 7**, a mature canvas library with a built-in model for selecting, dragging, resizing and rotating objects. Fabric has no official React bindings, so the editor drives it directly while a plain list of elements in React state stays the single source of truth that is saved.

Editor feature	How it works
Elements	Text (literal or a field), image (organisation logo or QR code), shape (rectangle or line) and icon (one of 18 built-in icons, or one from a Super Admin-managed uploaded pool). Up to 200 elements per design.
Dynamic fields	Trainee name, verb, credential title, date issued, certificate code, organisation name, signatory name and signatory title.
Page sizes	A4 landscape (1169×827), A4 portrait (827×1169), Letter landscape (1100×850), Letter portrait (850×1100). The design scale is about 100 units per inch and is scaled to fit when shown.
Undo and redo	A stack of up to 50 full snapshots, with keyboard shortcuts; each undo reloads the objects rather than patching, because an undo can jump across an add or a delete.
Snapping and guides	Within 6 units, an object snaps to other objects' edges and centres and the page centre, with dashed guide lines. Fabric has no built-in version of this, so it is computed by hand.
Multi-select and duplicate	Shift-click or box-select moves a group; right-click duplicates.
Autosave	Saves automatically with a guard that compares against the last saved design.
Background upload	JPG, PNG or WEBP up to 10 MB, only from the platform's own file storage; the page resizes to the picture's shape and, if the page is empty, five fields are laid out ready to drag.

WRITTEN RATIONALE

`CertificateCanvasEditor.tsx`: "Imperative, not declarative (Fabric has no official React bindings the way Konva's react-konva does)"; the React element list, "never Fabric's own internal object graph", is the single source of truth. Comments record real fixes: Fabric v6+ origins pinned to top-left because a 968-unit rectangle at x 16 once rendered at x −468; the canvas is scaled with a CSS transform rather than Fabric's own sizing because of a horizontal-offset quirk.

NO WRITTEN RATIONALE

No document explains why Fabric.js was chosen over other canvas libraries.

ENGINEERING INFERENCE

Fabric already provides the drag, resize, rotate and multi-select behaviour that would take months to build, and the saved design is plain data, so the library could be replaced later.

IMPORTANT

Accessibility limit stated by the team: the Fabric canvas “has not been checked for keyboard operation. Assume a person who cannot use a mouse cannot design a certificate there” (`docs/accessibility-testing.md`). The editor also has no contrast checker.

How a design is stored: bounded data, never markup

A design is saved as JSON validated by a zod schema: every value is a bounded number, a short string or one of a fixed set of choices (font size 6 to 200, colours as `#RRGGBB`, positions within limits, at most 200 elements). There is no HTML anywhere in the design, so there is nothing to clean: the schema is the safety boundary. The renderer builds style objects, never strings of HTML.

WRITTEN RATIONALE

`certificateLayout.ts` : “there is no markup anywhere in this shape, so unlike the HTML-import approach this replaces, there is nothing to sanitize: CertificateLayoutSchema’s own Zod validation IS the safety boundary.” The layout is “canvas-library-agnostic”.

Why a certificate never changes after it is issued: the design snapshot

When a certificate is issued, the template’s design (layout, logo address, signatory and organisation name) is copied onto the certificate itself as a versioned snapshot. Re-assigning a course to another template, editing templates or removing a logo from the picture bank can never alter a certificate already handed out. “No template” is also a real recorded state: it means “issued with AAICBI’s own default design”. Logos used by approved templates cannot be deleted for the same reason.

WRITTEN RATIONALE

`certificateDesignSnapshot.ts` : certificates used to be drawn live from whatever template the course pointed at, “so re-assigning a course to another template … silently re-designed every certificate already handed out. Snapshotting the design onto the Certificate row makes an issued certificate immutable the same way its code already is.”

Approval: the organisation signs off its own certificate

Step	What happens
1. Create and edit	The organisation (or a Super Admin) creates a template and designs it. A review code `CERTTPL-XXXX-XXXX` is generated immediately.
2. Send for review	The organisation’s contact receives an email with a link to a public, no-login preview showing a sample certificate (“Jane Doe”, “Sample Course”).
3. Approve	The contact approves with one click. The approval is a conditional update (“only if not yet approved”), so a double click or a replayed link cannot approve twice. The approve link is rate limited to 20 an hour per address.

Step	What happens
4. Locked	An approved template can no longer be edited, deleted or have its logo changed. Only approved templates can be assigned to a course.

WRITTEN RATIONALE
Approved templates are locked for "the same immutability the certificate code itself already has once issued".

ENGINEERING INFERENCE
A token link lets a signatory who has no platform account sign off the design, and organisations approve their own branding before it is used.

Issuance, the code and public verification

- **When:** automatically, the moment a trainee passes the course examination (earlier versions issued at 100% module completion; older certificates were revoked by a one-off script rather than quietly kept, because two definitions of "AAICBI certified this person" would be worse).
- **Race-safe:** creation is "insert and ignore a duplicate"; an old row from the previous rule is upgraded in place with a conditional update. Issuance never throws, so a certificate problem can never affect exam submission.
- **Code:** `AAICBI-XXXX-XXXX` — eight random characters from an alphabet of 31 that leaves out look-alikes (no 0/O, 1/I/L), so people can copy it by eye.
- **Verification page** `/certificate/<code>` needs no sign-in, is limited to 60 lookups an hour per address, shows only the name, course, dates and code (never email or account details) and has three states: not found, revoked, valid. A QR code on the certificate points to it.
- **QR:** made on the server with the `qrcode` library as a PNG, offline and with no outside service, so there is nothing to configure or rate limit.
- **Revocation:** instructors or admins who own the course can revoke or restore; the public page reads the state live, so a revocation takes effect at once.
- **Printing:** "download" is the browser's Print / Save as PDF on the same page, described in the code as "a deliberate scope decision, not an oversight".

WRITTEN RATIONALE
Schema comments: authenticity is a database lookup, not a cryptographic signature, because key management, a verification algorithm and key rotation would be "real added complexity", and "a corrected database is more honest than an immutable one for a training certificate", so revocation can work. The code alphabet avoids "characters people reliably mis-transcribe". QR as a PNG avoids needing `dangerouslySetInnerHTML` in the render path.

The "Powered by AAICBI" mark

The renderer draws "POWERED BY AAICBI.ORG" at the foot of the page itself, after the design elements, so nothing in a saved design can remove it. Removing it is a separate monthly Paystack

subscription per organisation, or a Super Admin waiver. The state is read live, not frozen, because it is billing state. The default AAICBI design always carries it.

WRITTEN RATIONALE

"There's nothing in layoutJson an org could strip via the editor or a raw API call to remove it." The watermark subscription is mirrored from platform-fee billing "rather than generalized … so a change to platform-fee billing can never silently also change watermark-removal billing's behavior".

Why certificates are drawn as HTML and not as server PDFs

Certificates are shown on the public page as scaled HTML and CSS: the design is rendered in a fixed design-size box which is scaled to the screen by a resize observer. Fonts are ordinary CSS font names (for example Georgia or Helvetica), so they depend on the viewer's system fonts. The team calls the absence of a server PDF pipeline for certificates a deliberate scope decision.

NO WRITTEN RATIONALE

The reason for choosing browser printing over a server-made certificate PDF is not written down beyond "deliberate scope decision".

ENGINEERING INFERENCE

It avoids shipping and licensing fonts for every organisation's design and keeps one rendering path for the page, the preview and the print. The trade-off is that exact typography can vary by device.

Standalone exam certificates

Standalone exams use a separate `ExamCertificate` record with the same code generator and the wording "has successfully passed". They were kept apart from course certificates because forcing them together "would mean making Certificate.courseId nullable and bending its heavily race-condition-audited issuance logic".

Every other document engine

The platform makes nine other kinds of document or image, each with the simplest tool that fits.

Output	Tool	Notes
Curriculum outline PDF	@react-pdf/renderer	Made on demand from the current course; public for visible courses.
Assignment submission PDF	@react-pdf/renderer	Emailed to the instructor when AI marking is off; not stored.
Performance export	@react-pdf/renderer	Course KPIs and a trainee table.
Instructor agreement	@react-pdf/renderer	Letter text plus a signature image embedded as text so it cannot go missing on the host.

Output	Tool	Notes
Trainee learning report	@react-pdf/renderer	A trainee's own activity report.
Admin analytics report	@react-pdf/renderer	Downloadable and sent weekly; stat grids and tables, no chart library.
Growth share PDF	@react-pdf/renderer	One page; the whole card is a link to the course funnel.
Word to PDF	mammoth, node-html-parser, @react-pdf/renderer	Word is turned into HTML, parsed into a small set of document nodes (headings, paragraphs, lists, images, tables), then drawn; cached by file version.
Result card images	next/og	1200×630 and 1080×1920 PNG; edge cached.
Animated promo card	Playwright and ffmpeg (an offline script)	MP4/GIF/PNG/WebP files built before release, not at run time.
Picture shrinking	sharp	Turns stored pictures into WebP (quality 80) at 480, 1280 or 1600 px; replaced only if at least 15% smaller; the old file is deleted only if nothing still uses it.

WRITTEN RATIONALE

`curriculumPdf.tsx` : react-pdf "was chosen specifically because it's pure JS/Node (no headless-browser binary like puppeteer+@sparticuz/chromium would need), well within a standard Vercel serverless function's size/time budget, and the standard tool for exactly this 'structured document from data' case." PDFs are made on demand so there is "no cache-invalidation problem". For Word to PDF: "No new heavy dependency, no headless-browser binary, no third-party API."

WRITTEN RATIONALE

`next.config.js` records a real production bug: PDF font files were missing from the Vercel bundle ("Cannot find module … Helvetica.cjs"), fine locally and broken on the host, fixed once for all PDFs with a file-tracing include.

Reading documents in

Kind	Tool	Rules
Word (.docx)	mammoth	Raw text, then plain pattern splitting into questions, then AI structuring; uncertain items are flagged for a person.
Word (.doc)	word-extractor	Older format needs its own reader.
PDF	unpdf	Text PDFs only; scanned PDFs are refused with a clear message.

Kind	Tool	Rules
PowerPoint and Excel	jszip	Opened as zip files; slide and note text and sheet text are read from the XML.
Detection	file bytes (magic numbers)	The type comes from the bytes, “not its name, which can lie”; text is capped at 60,000 characters.

WRITTEN RATIONALE

`docxParser.ts` : the splitter is “deliberately dumb and regex-based … anything genuinely ambiguous is exactly what the AI structuring step and the human review screen are for”.

Why the Certificate Studio design suits AAICBI

- **Branded and trusted.** Organisations design their own certificates, approve them themselves, and every certificate can be checked by anyone with a code or a QR.
- **Immutable once issued.** Snapshots, locked templates and unique codes mean a certificate means the same thing tomorrow as today.
- **Safe by construction.** Bounded data, no markup, an allow-list for backgrounds, rate limits on public pages and an unremovable footer.
- **No heavy infrastructure.** No headless browser, no external QR or PDF service, no font licensing.

LIMITATION

Gaps recorded by the research: there are no automated tests for certificate issuance, the watermark logic, the editor, the renderer or the PDF output, and a few comments are stale (for example one says the old default card “was removed”, but it is still the fallback for templates with no layout).

8 The learning engines

Exams, assignments, course progress, early warning and analytics. Each one is a small set of rules that can be tested on their own.

The exam engine

An examination is a timed, graded set of questions. The engine lives in `examEngine.ts` (the part that talks to the database) and `examEngineCore.ts` (the part that only does arithmetic and ordering). Three rules are written at the top of the file because they are easy to break in a hurry:

- **The answer key never reaches the browser during an attempt.** One function, `serveableQuestion()`, is the only code allowed to shape a question for the client before submission. It leaves out `isCorrect` and `explanation`.
- **The timer is cosmetic on the phone.** `expiresAt` is calculated once, on the server, when the attempt starts, and every later write is checked against it. Changing the device clock does not add time.
- **Grading happens once, atomically.** Submitting an attempt that is already submitted returns the existing result and does not grade again.
- **Starting is idempotent.** A double tap on "Start" or a reload of the instructions page does not burn a second attempt; the unique key on exam, trainee and attempt number catches the race, and the engine returns the attempt already in progress. The maximum number of attempts is enforced here.
- **The question order is saved** with the attempt, so a reload shows the same order. `reconstructOrderedItems` rebuilds it from the saved list.

> **WRITTEN RATIONALE**
> `examEngine.ts` header: "never send `isCorrect` or `explanation` to the browser for a question in an IN_PROGRESS attempt … the timer is cosmetic on the client … grading happens once, atomically, on submit." A comment records the "M11 audit finding" that led to the double-tap protection.

> **ENGINEERING INFERENCE**
> Postgres transactions and unique constraints are the right tool for "exactly once" because they work the same on one server or fifty serverless instances, which a lock held in memory would not.

Performance analysis after an assessment

When a trainee submits, `performanceCore.ts` groups the answers by topic and counts right and wrong, with no AI at all. Questions without a topic go in a "General" bucket rather than being dropped, so nothing is left uncounted. Only then does a small AI call (`analyzePerformance.ts`) turn those numbers into a short narrative. The AI sees only the numbers, so it has nothing to make up.

WRITTEN RATIONALE
`analyzePerformance.ts` : an audit found it had been using the larger model; “claude-haiku-4-5 is the deliberate choice now — cheaper, faster” because it runs on every submission and does a simple job.

The assignment engine

Assignments take written answers. When a trainee submits, the submission is first marked SUBMITTED atomically; then `assignmentGrading.ts` decides what happens next. If AI assessment is on for the assignment, each answer goes to Claude **one at a time** (for predictable rate-limit behaviour) with the question, the rubric, the expected concepts and the answer. If it is off, no AI call happens at all: the whole submission is rendered as a PDF and emailed to the instructor for marking.

- **Meaning, not wording.** The prompt tells the assessor to judge whether the student shows understanding, not whether the words match.
- **Structured, validated reply.** The model must return JSON with a score, maximum, per-criterion scores and reasons, strengths, areas to improve, feedback, a confidence and a `needs_instructor_review` flag. The reply is validated with zod; the score is clamped, because the code never trusts the model’s arithmetic.
- **The stronger model for the grade.** Grading uses `claude-sonnet-4-5` because the grade is relied on by the student and the instructor, so quality matters more than the cost per call.
- **Low confidence goes to a person.** A low-confidence answer is flagged for the instructor instead of being final.
- **Paste is blocked** in answer boxes, so answers are written by the trainee.
- **A resubmission path and a submission PDF** (`@react-pdf/renderer`) exist for instructors who mark by hand.

WRITTEN RATIONALE
`assessAssignmentAnswer.ts` : “claude-sonnet-4-5 deliberately, not haiku — this produces an actual grade a student and instructor both rely on, so quality matters more than per-call cost.”
`assignmentGrading.ts` : “sequential, not parallel, for predictable rate-limit behavior.”

LIMITATION
Grading guidance cannot be trained. The “Train the AI” screen lists “Grading assignment answers” as deliberately not trainable, because guidance could bias marks (see chapter 9).

Courses, modules and progress locking

A course is made of modules, a module of lessons and an assessment. `progressCore.ts` decides, from plain numbers, whether a module is complete and which modules are open. A module is complete when all its lessons are done and, if it has a **published** assessment, that is passed. The next module unlocks only when the one before is complete. `findNextModule` picks where “Continue” should land and `isCourseComplete` decides when a certificate may be issued. `progress.ts` fetches the numbers and calls the core.

ENGINEERING INFERENCE

Keeping “what counts as complete” in one pure function means the trainee page, the organisation dashboard, the certificate engine and the early-warning rules all agree, and the rule can be tested with a table of cases instead of a browser.

Early warning for staff

`earlyWarningCore.ts` raises two kinds of alert for a course: **inactivity** (a trainee who was active has not logged in for the course’s threshold of days) and **failed attempts** (too many failures). Two details show the care taken:

- A trainee who has **never** logged in is not counted as inactive; that is a different problem (never started), and the code says so as “a conscious scope line”.
- An alert fires **again only when something has changed**: the trainee came back and went quiet again, or passed since the last alert. “Time passed” is not enough, so staff are not told the same thing every day.

WRITTEN RATIONALE

`earlyWarningCore.ts` : the INACTIVITY and FAILED_ATTEMPTS re-trigger rules are written out with their reasoning; “Not an oversight; a conscious scope line.”

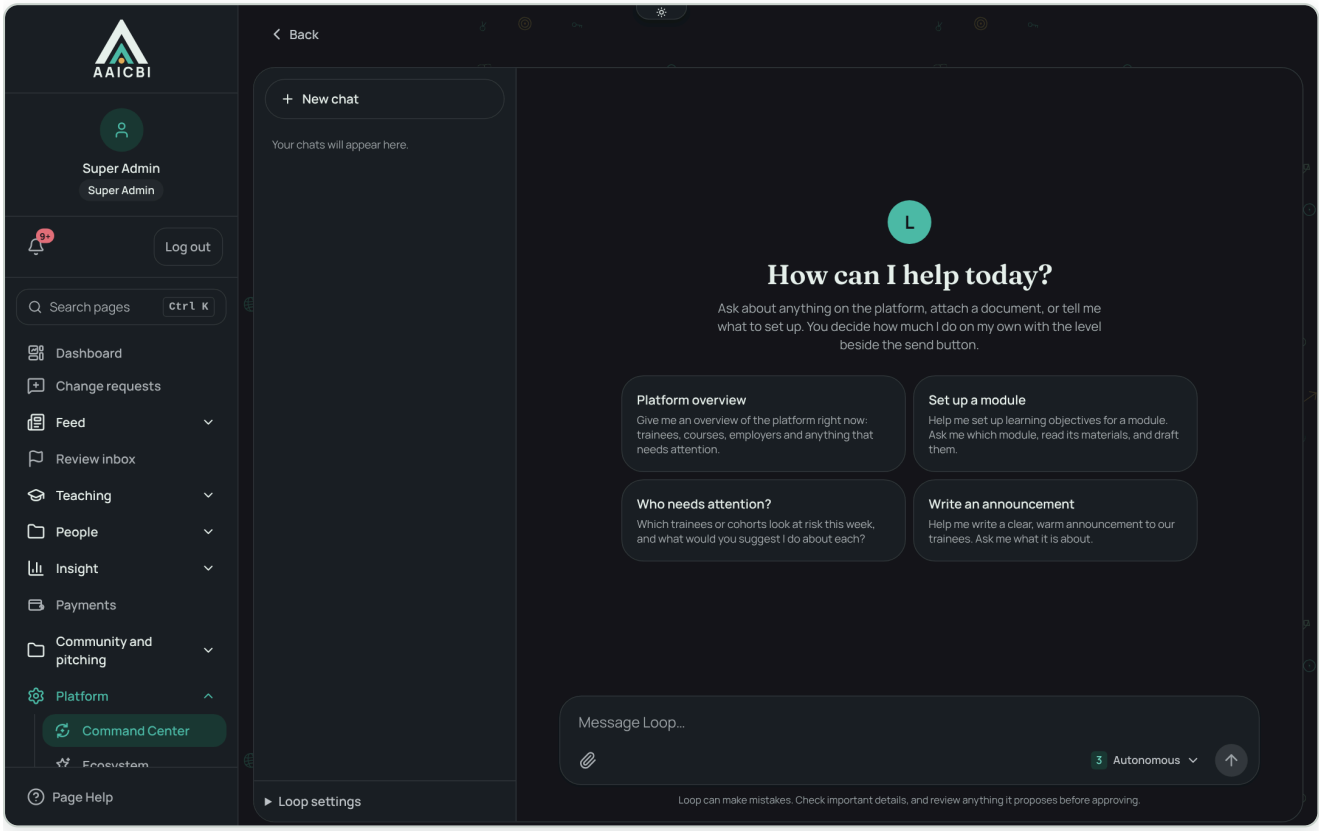
Analytics, reports and the daily crons

The analytics code (`src/lib/analytics`) aggregates visits, enrolments, completion and engagement into cohorts, streaks, segments and interest scores. Five scheduled jobs on Vercel keep the platform tidy; each is an ordinary API route protected by a secret:

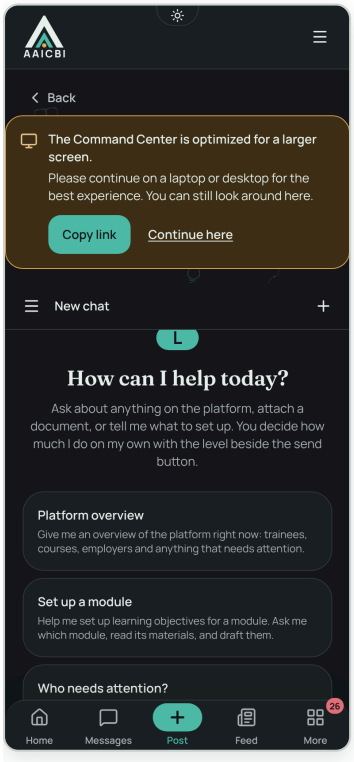
Cron route	When (UTC)	What it does
`/api/cron/arena-sweep`	daily 05:00	Settles Arena invitations that ran out unanswered (and tells the sender). The app does this within seconds while people are online; the daily job is the backstop for a quiet day.
`/api/cron/course-access-sweep`	daily 06:00	Handles lapsed course access and expiry reminders; the platform’s first scheduled job.
`/api/cron/analytics-alerts`	daily 07:00	Checks the early-warning rules and writes staff alerts. Kept as its own job, separate from the access sweep.
`/api/cron/analytics-report`	Mondays 08:00	Builds and sends the weekly analytics report as a PDF.
`/api/cron/growth-messages`	every hour	Queues and sends follow-up messages that have come due for people who saved a taster score. Does nothing while messaging is off and sends nothing in quiet hours.

ENGINEERING INFERENCE

Scheduled routes on the host's own scheduler avoid running a separate worker server. The cost is that a job must finish within a serverless time limit, which is why each one does a bounded amount of work per run and is safe to run twice.



COMPUTER



PHONE

Figure 11. The Command Center, where a Super Admin reads the platform's reports and talks to Loop.

9 The AI layer

Claude is used in a dozen places. Each use has a model, a safeguard and a plain reason. This chapter lists them all.

Why Claude, and why through one small client

Every AI feature calls Anthropic's Claude models through the official `@anthropic-ai/sdk`. The platform follows one house convention, written at the top of several AI files: **one file per feature, a system prompt that states the exact rules, a JSON reply validated with zod, fences stripped before parsing, and one retry before giving up.** If the key is missing or the call fails, the feature says so and the rest of the platform keeps working.

> ENGINEERING INFERENCE
> Claude follows long, rule-heavy instructions closely and returns structured output reliably, which matters when the output decides a grade or a published question. Using one vendor, one SDK and one pattern keeps the security review small: there is one key, one billing account and one place to look.

Which model does what

Feature	Model	Why this model
Question extraction from uploaded documents (`extractQuestions`)	claude-sonnet-4-5	Reading messy documents and never inventing an answer needs the stronger model. Two attempts at most.
Assignment question extraction	claude-sonnet-4-5	Same reasoning; the model is told explicitly not to guess a field it cannot find.
Grading an assignment answer	claude-sonnet-4-5	A real grade; "quality matters more than per-call cost".
Performance summary after an assessment	claude-haiku-4-5	Runs on every submission and only rewords numbers: cheaper and faster.
Learning patterns across assignments	claude-haiku-4-5	Same shape as the performance summary; fed only the numbers.
Question bank writer	claude-sonnet-5-5 (default)	Drafts questions from lesson material. Configurable with `BANK_GENERATOR_MODEL`.
Question bank reviewer	claude-opus-5-5 (default)	A different, stronger model re-derives every answer blind. Configurable with `BANK_REVIEWER_MODEL`.

Feature	Model	Why this model
Guide consultant (advice on Loop's knowledge)	claude-sonnet-4-5 (default)	Advice only; never writes an answer by itself. Configurable with `GUIDE_CONSULTANT_MODEL`.
Job-posting screening	claude-sonnet-4-5	Looks for scams and discrimination before a posting goes live.
Learning Buddy, Command Center, pitch feedback	Claude through the Loop engine	Conversation with tools; see below.
Course examination generation, learning objectives	claude-sonnet-4-5	Drafts that a person must approve.

WRITTEN RATIONALE

`bankModels.ts`: "One model WRITES the questions and a DIFFERENT model REVIEWS them, so a question is never checked by the model that wrote it (the same model tends to repeat its own mistakes)." The same file explains that setting both roles to the same model weakens the bank.

LIMITATION

Model names are read from the code on the publication date. Model versions change; the environment variables above let the Super Admin move to a newer model without a code change for the bank and the consultant. The other features have the model written in the file.

Safeguards that apply to every AI feature

- **A person approves anything that is published.** Generated questions are drafts until reviewed; the consultant's suggestions are cards a Super Admin approves through the normal form; Loop's actions wait for approval according to the chat's autonomy level.
- **The model only sees what it needs.** Performance feedback gets numbers, not the transcript. Assignment grading gets one question and one answer.
- **Never invent.** Extraction prompts forbid guessing an answer or a field.
- **Validated output.** zod schemas reject a reply that does not match; scores are clamped.
- **Timeouts.** Calls carry a timeout (30 s for grading, 60 s for the consultant) so a slow call cannot hang a page.
- **Graceful absence.** No key means the feature explains that it is unavailable.

Training the AI, safely

A Super Admin can approve a short guidance note for six features: the Command Center assistant, Learning Buddy, pitch feedback, assignment feedback summaries, performance summaries and question writing. The note is plain text, versioned, and added **after** the feature's own rules so it can never replace or relax them. Three things are deliberately **not** trainable because they decide

outcomes or protect people: grading assignment answers, checking the answer key of generated questions, and screening job postings.

> **WRITTEN RATIONALE**
> `aiGuidanceCore.ts`: "It is added AFTER the feature's own rules and can never replace or relax them"; the `NOT_TRAINABLE` list gives the reason for each exclusion ("Guidance could bias marks.", "It must stay independent.").

Loop and the Command Center

Loop is the assistant character on every page. For trainees it is a learning buddy; for the Super Admin the Command Center is a full conversation engine (`loop/engine.ts`) that streams Claude's answers and lets Claude read real platform data and documents through **tools**. Everything Claude can change goes through an **action registry** (`loop/actions.ts`); the chat's autonomy level (`policy.ts`) decides whether an action runs now or waits for approval, and each run is recorded. The code states the principle: Claude's own ability to discuss and plan is not narrowed; what it can **do** is exactly the tools and actions listed.

> **WRITTEN RATIONALE**
> `loop/engine.ts` header: "Claude's own capacity is not narrowed here … What it can DO is exactly the tools and actions below."

The guide engine: answers without AI

Before any AI is involved, Loop answers most questions from a curated knowledge base, matched by plain text rules (`guide/match.ts`, `intent.ts`, `similarity.ts`). It understands intents, offers "Take me there" and "Show me" buttons, respects the asker's role, and can run a highlighted tour of the screen (`data-guide-target` marks). Unanswered questions go into a review queue ranked by priority; a Super Admin writes the answer once and everyone benefits. A test enforces that every page in the platform is registered in the site map.

> **ENGINEERING INFERENCE**
> Answering common questions from approved text is free, instant, consistent and cannot hallucinate. The AI consultant is held back for the queue and for advice, which is where it adds value.

Monitoring and the consultant

`monitorCore.ts` builds a snapshot of the platform (analytics, materials, assessments, performance) and applies deterministic signals. The consultant (`guide/consultant.ts`) turns the queue, the snapshot and module materials into suggestions, a platform report and proposals for AI-training notes. It runs only when a Super Admin presses a button **and** the switch is on, uses the platform's existing Anthropic key, and its usage is audited.

Embeddings and duplicate detection

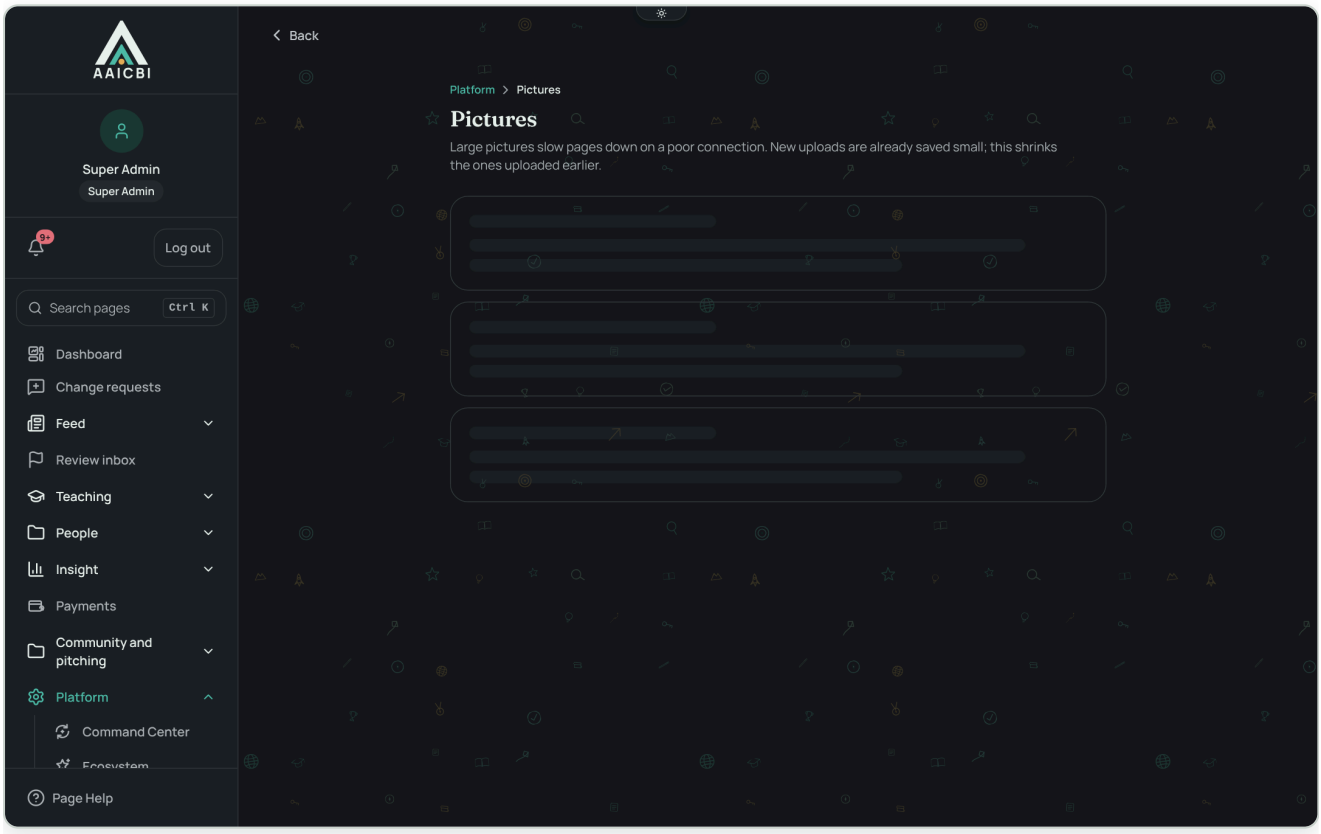
Imported questions are compared by meaning so the bank does not fill with near-copies. Voyage's `voyage-3-lite` produces a 512-number vector for each question; the vector is stored in Postgres using the `pgvector` extension (`vector(512)`), and `embeddingsCore.ts` applies cosine similarity with a duplicate threshold of **0.93**. The maths is in a zero-import file so it can be tested; the database lookup is separate.

WRITTEN RATIONALE

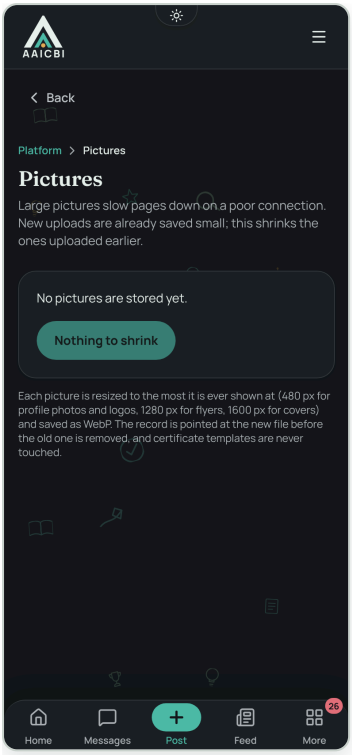
`embeddingsCore.ts` explains that the pure maths is split from `embeddings.ts` so tests need no Prisma client or API key; `embeddings.ts` expects a dimension of 512.

ENGINEERING INFERENCE

Storing vectors in the same Postgres avoids a second database to buy, secure and back up. A small, cheap embedding model is enough for "is this the same question reworded?".



COMPUTER



PHONE

Figure 12. The Platform pictures screen, the maintenance tool that shrinks oversized pictures (chapter 10).

10 Search, social, messaging, money and speed

The supporting engines: finding things, posting safely, reaching people, taking payment and working on a poor connection.

Search

Global search covers people, organisations, employers, investors, startups, jobs and courses, with filters and suggestions. It uses Postgres itself: a search-indexes migration adds trigram (`pg_trgm`) and ordinary indexes for typo-tolerant matching and filters. Results respect visibility: a profile marked private does not appear.

> ENGINEERING INFERENCE
> At this scale, search inside the existing database is faster to build, always consistent with the data and has nothing extra to run. A dedicated search server would add cost and a sync problem for little gain.

The social feed and automatic review

Posts from members, showcase projects and organisation news share one `Post` model with one review pipeline. `posts/autoReview.ts` adds up warning signs in the words of a post (abuse, scam phrases and similar) and gives a score. Below the bar the post publishes at once and the decision is written to its history as "Auto-review", so a person can still remove it later; at or above the bar it goes to the human review inbox with the reasons attached. **Nothing here ever rejects a post: only people reject.** The bar is **2** for ordinary members and **5** for trusted authors (a track record, nothing taken down, nothing open against them). Setting `POST_AUTO_REVIEW=off` sends everything except a Super Admin's posts to a person.

- **Reviewers are a permission**, not a role: staff with the review permission see a unified queue; serious cases escalate to the Super Admin; an organisation can be blocked from posting.
- **Public feed with an account gate:** anyone can read; interacting needs an account.
- **Handles:** every person has one address (`/@handle`). Old handles keep working after a change, and changes go through a request.
- **Showcase sync** mirrors a trainee's project into the feed, through the same review.

> WRITTEN RATIONALE
> `autoReview.ts` header: "Nothing here ever rejects a post: only people reject … Trusted authors … are held to a higher bar." The word lists are "short on purpose: they catch the obvious, and everything else is left to the reports and to the people who review."

LIMITATION

The automatic reviewer is a word-and-pattern scorer, not an AI model. It will miss anything that does not use the listed words, which is why reports and human reviewers remain part of the design.

Notifications, email, push and WhatsApp

Channel	Tool	How it is used	Why it fits
In-app	Postgres rows	A notification list and unread count on every page.	Always available; no vendor.
Email	Resend	Account, password, reminders, reports. Templates in `notifications/templates.ts`.	Simple API; sending only.
Web push	`web-push` with VAPID keys and the service worker	Opt-in per device. At most 10 subscriptions per account. Urgent messages stay on screen and vibrate.	Reaches a phone that has the browser closed, with no app store.
WhatsApp	Meta WhatsApp Cloud API	Template messages only, because Meta only allows a business to start a chat with an approved template. Credentials and the switch are kept by the Super Admin on the Growth page; off until switched on.	WhatsApp is where many Nigerian learners already are.

WRITTEN RATIONALE

`whatsapp.ts` states the template-only rule, that it is tested against a stand-in server that answers like Meta's API, and that it "has NOT been run against Meta itself: that happens the first time real credentials are saved and 'Test connection' is pressed."

IMPORTANT

WhatsApp delivery has not been proven against Meta's live service. Treat it as built and simulated until the first real test message succeeds.

Messaging between people

Trainees, organisations, employers and staff can message each other. A policy file decides who may start a conversation with whom, a moderation step checks messages, broadcast messages resolve their recipients from a filter, and a new message triggers a notification and, if allowed, a push.

Payments, billing and seat limits

- **Paystack** takes card and bank payments. The webhook route is the only door for Paystack and authenticates by **signature**; the payload is redacted before it is stored; a confirmed charge is processed once by `processConfirmedCharge`; subscription events are matched to the right enrolment or organisation.

- **Organisation billing:** a DIRECT_PAYMENT organisation pays a platform fee and a certificate-watermark fee; a Super Admin can also confirm a payment by hand.
- **Seat cap:** `trainingOrgSeatCap.ts` counts every trainee with real, active access to any course of the organisation, however they got it. A free preview is a sample and never uses a seat.
- **Sponsored ads:** `adsCore.ts` defines what an ad may contain (headline 60, caption 140, button 24 characters), when it is live (Nigerian time, WAT), and how views and taps are counted; placements are the top of the feed, inline after every 6 posts, and the About page.
- **Growth ledger:** `growthCore.ts` keeps the public taster to five questions, 18 seconds each, and shows a percentile only after 30 people have finished so a small crowd never produces a made-up claim.

> ENGINEERING INFERENCE
> Paystack is the payment provider that supports Nigerian cards, transfers and subscriptions out of the box. Signature-first webhooks and “process once” are the standard way to avoid double-crediting.

Speed, low bandwidth and the installable app

- **Progressive Web App.** A manifest, icons and a service worker (`public/sw.js`) make the site installable. It caches the shell and static files, never intercepts `/api/` calls, and shows an offline page. The install prompt appears only after a second visit and not within 30 days of “Not now”; iPhone gets Share instructions because Safari has no install button.
- **Lite connection mode.** `useLiteConnection` starts on, so the first paint never starts a large download, and turns off on a good connection. On 2G, 3G or data-saver, videos do not download by themselves.
- **Picture shrinking.** `sharp` shrinks oversized stored pictures to the width each kind is ever shown (480 px for avatars and logos, 1280 for course flyers, 1600 for covers). A new file is used only if it is at least 15% smaller, and small files are left alone.
- **Server-rendered pages** with Tailwind CSS keep the JavaScript sent to phones small; heavy libraries (the certificate editor, charts) load only on the pages that use them.
- **Phone-first layouts.** A role-aware bottom navigation, tables that turn into cards, 16 px inputs with proper keyboards, step-by-step long forms and a draggable Loop that never blocks a button.
- **Languages.** Interface strings can be translated; Yoruba, Igbo, Hausa and French are drafted with Google Translate and Nigerian Pidgin with Claude, because Google does not support Pidgin, and every draft is reviewed before it is trusted. English is the fallback.

> WRITTEN RATIONALE
> `useLiteConnection.ts`, `pictures/core.ts`, `pwa/installCore.ts` and `i18n.ts` each state their rule and reason in comments; the Pidgin-by-Claude choice is written in the schema comment for the translation model.

Landing, About and public pages

The home page is the public feed; the former landing page is `/about`. Public pages for jobs, trainees and organisations are server-rendered and indexable, and use rounded “social” styling with a mint-to-gold card gradient and the triangle logo. Staff sign in on a separate, configurable address that is not linked from public pages.

11 Quality engineering and deployment

How the team knows the engines behave: tests that need no database, real-browser checks, accessibility sweeps and a build that migrates safely.

The “Core” pattern, and why it matters most

The most important engineering habit in the repository is splitting each engine in two. A **Core** file holds the rules as pure functions: no database, no network, no AI, nothing imported. A second file does the fetching and saving and calls the core. The repository has 55 such core files, among them `examEngineCore`, `progressCore`, `earlyWarningCore`, `rateLimitCore`, `embeddingsCore`, `arenaCore`, `adsCore`, `growthCore`, `autoReview`, `pwa/installCore` and `rbac` rules.

- **Fast tests.** A rule can be tested with a table of inputs in milliseconds.
- **Honest tests.** The test calls the same function the server calls; there is no copy of the rule in the test.
- **One place to change.** The trainee page, the organisation dashboard and the certificate engine cannot disagree about whether a module is complete.

> **WRITTEN RATIONALE**
> The split is explained in the headers of `rateLimitCore.ts`, `embeddingsCore.ts`, `examEngineCore.ts` and `progressCore.ts`. `rateLimitCore.ts` records that the split came from “a real bug caught while actually running the tests”: importing the pure function had pulled in a database client that could not start in the test environment.

The layers of testing

Layer	Tool	What it covers	Where
Unit and rules tests	Vitest	About 1,360 test cases in 144 files: engines, permission rules, grading, formats, templates, schema guards.	`tests/`, run with `npm test`
Real-browser checks	Playwright driving Chromium	111 scripts that sign in as real demonstration accounts on a local database and click through flows on an emulated iPhone, Android phone, tablet and desktop.	`scripts/device-checks/`
Accessibility sweep	axe-core through Playwright	Scans a list of pages for accessibility problems.	`scripts/a11y-sweep.mjs`, `npm run a11y`

Layer	Tool	What it covers	Where
Type checking	TypeScript strict	Every build fails on a type error, including misuse of a notification type or a permission key.	`next build`
Lint	ESLint through `next lint`	Style and common mistakes.	`npm run lint`

ENGINEERING INFERENCE
Rules tests give fast, exact feedback on the parts where a mistake costs the most (grades, access, money). Real-browser checks cover what unit tests cannot: the layout on a phone, a keyboard path, a redirect. Using the same Chromium that makes the user manuals keeps one browser toolchain.

LIMITATION
The repository contains no continuous-integration workflow files (there is no `.github/workflows` folder). Tests are run by the developer and by the hosting build, not by an automated pipeline on every push. This is listed in chapter 13.

Database changes: migrations

The schema lives in `prisma/schema.prisma` with 178 models and enumerations changed through 109 migrations. Each migration is a reviewed SQL file. Extensions needed by features (`vector` for embeddings; trigram for search) are created inside migrations. A custom build script (`scripts/vercel-build.mjs`) runs the build on Vercel.

Deployment

- **Hosting:** Vercel builds the Next.js app and runs it as serverless functions; five scheduled jobs are declared in `vercel.json`.
- **Database:** Neon Postgres, with pooled connections suited to serverless.
- **Files:** Vercel Blob for uploads such as pictures, certificates' backgrounds and documents.
- **Secrets:** environment variables hold the Anthropic, Voyage, Ably, Resend, Paystack, VAPID and cron keys. Each optional service is checked at the moment it is used, and the Super Admin can see integration status on a screen.
- **Squash merges** to `main` through pull requests keep the history readable.

ENGINEERING INFERENCE
A single hosting vendor with a managed Postgres lets a small team ship without operating servers, and per-use pricing suits a platform that starts small.

What “graceful degradation” means in practice

If this is missing	What still works	What stops
Anthropic key	Everything except AI features; the guide answers from curated text.	AI grading, drafting, summaries, consultant.
Voyage key	Questions can still be imported.	Meaning-based duplicate detection.
Ably key	Arena and chat, by polling every 1.5 s.	Instant updates.
Resend key	In-app notifications.	Email.
VAPID keys	Everything else.	Phone push.
WhatsApp credentials	Email and in-app messages.	WhatsApp messages (off by default).
Paystack	Free courses and manual payment confirmation.	Online card payment.

ENGINEERING INFERENCE

Each service sits behind a small function that checks its own configuration. This follows from the Postgres-only design: the platform needs only the database to run, and everything else adds capability.

12 The decision register

Every major choice on one page each: what was chosen, what else could have been used, the reason, and how well the reason is documented.

Read the last column carefully. **Written rationale** means the reason is recorded in the code or documents. **Engineering inference** means we are explaining why the choice fits, without a note from the original author. **No written rationale** means we found nothing written.

Decision	Chosen	Common alternatives	Why it fits	Evidence
Web framework	Next.js 14 App Router, React 18	Separate API and single-page app; Remix; Laravel	One codebase for pages and API, server rendering for speed on weak phones and for search engines.	Inference
Language	TypeScript, strict	JavaScript	The compiler catches misuse of permission keys, notification types and database models before release.	Inference
Database	PostgreSQL 16 through Prisma 5	MongoDB; MySQL; Firebase	Relational data with many links; transactions and unique keys give “exactly once”; vectors and fuzzy search live in the same database.	Inference; extension use is written
Database host	Neon	Self-hosted Postgres; Supabase; RDS	Serverless pooling and branching suit Vercel.	Inference
Hosting	Vercel (with Blob and crons)	A virtual server; AWS; Render	No servers to run; the app and its schedules deploy together.	Inference

Decision	Chosen	Common alternatives	Why it fits	Evidence
Styling	Tailwind with CSS-variable brand tokens	Component libraries	Small CSS, one dark and light theme by changing variables.	Inference
Sessions	Signed cookie with `jose` (HS256), bcryptjs cost 10	Third-party sign-in; JWT in local storage	No external dependency; the cookie is not readable by scripts.	Inference
Permissions	Deny-by-default `category.feature:action` with templates and audit	Fixed roles only	Organisations need different staff powers; anti-escalation stops a person granting more than they hold.	Written (RBAC documentation)
Rate limiting	Postgres-backed window limiter	Redis	No extra service; the same database already exists.	Written (`rateLimitCore.ts`) for the split; inference for Postgres
Server fetch safety	SSRF guard resolving DNS first	Checking the URL text only	A hostname can point at an internal address; the guard checks the resolved address.	Written (`ssrfGuard.ts`)
Game formats	Quiz, matching, ordering, scenario with pure grading functions	One generic question type	Each format has different cheating risks and scoring; each can be tested alone.	Inference; see chapter 5
Question quality	Writer and reviewer are different models	One model checks itself	The same model repeats its own mistakes.	Written (`bankModels.ts`)
Real time	Ably publish-only from the server, SSE in the browser, polling fallback	Own WebSocket server; Firebase	Server decides, vendor only nudges, nothing private travels on the channel.	Written (`realtimeCore.ts`)

Decision	Chosen	Common alternatives	Why it fits	Evidence
Match fairness	Server deadlines, versioned saves, private views	Trust the client	Cheating and double-tap races are closed on the server.	Written (`arenaCore.ts`, `arenaView.ts`)
Certificate editor	Fabric.js 7 canvas, saved as data	HTML/CSS editor; a fixed template	Free placement of text, images and QR codes with a layout that can be re-rendered as a PDF.	Inference; see chapter 7
Certificate output	@react-pdf/renderer, `qrcode`, `sharp`	Screenshots of a web page	Real PDF text, small files, QR code for verification.	Inference
Document reading	mammoth, word-extractor, unpdf, jszip, node-html-parser	A single converter service	Reads Word and PDF lessons inside the app with no outside service.	Inference
AI grading model	claude-sonnet-4-5	Haiku (cheaper)	A grade must be good.	Written
AI summaries model	claude-haiku-4-5	Sonnet	High volume, simple task.	Written
Embeddings	Voyage voyage-3-lite, `vector(512)`, cosine, threshold 0.93	Larger embedding models; text matching	Cheap and good enough for “same question reworded”.	Dimension written; model choice inference
Post review	Rule-based scorer; people reject	AI moderation; manual only	Instant for good posts, never rejects by itself.	Written
Email	Resend	SMTP; SendGrid	Simple sending API.	No written rationale
Payments	Paystack with signed webhooks	Flutterwave; Stripe	Nigerian cards and transfers.	Inference; signature approach written
Charts	Recharts	Chart.js; D3	React-native chart components.	No written rationale

Decision	Chosen	Common alternatives	Why it fits	Evidence
Icons	lucide-react	Icon fonts	Tree-shaken SVG icons.	No written rationale
Fonts	Fraunces and Manrope through next/font	Hosted font links	Self-hosted, no layout shift.	Inference
Testing	Vitest, Playwright, axe-core	Jest; Cypress	One toolchain for rules and browsers.	Inference
Offline and install	Service worker PWA	Native apps	No app store, one codebase, works on low-end phones.	Inference

TIP

When a reviewer asks “why was X chosen?”, the **Evidence** column tells you how firm the answer is. Where it says Inference, the honest reply is that the choice fits the requirement, not that the original author recorded this reason.

13 Open items and honest limits

What this guide could not prove, where the platform is simpler than it sounds, and what to check next.

Things built but not proven live

- **WhatsApp** has been tested against a stand-in server only; no real message has been sent through Meta.
- **Machine translation** drafts for Yoruba, Igbo, Hausa and French use Google Translate and Pidgin uses Claude; the code requires human review before trusting them, and this guide did not check how much has been reviewed.
- **Load.** The platform was read, not load-tested. The design choices (serverless, polling fallback, bounded cron jobs) suggest it will scale gradually, but no number is claimed.

Where the simple version is the real one

- The **post auto-reviewer** is a scorer over word lists, not an AI.
- The **guide** answers from curated text; AI is used only to advise the Super Admin.
- **Search** is Postgres trigram and indexes, not a search engine.
- **Rate limiting** uses the database, so it is shared across servers but adds a small database cost to each limited request.

Gaps to consider

- **No continuous integration.** Nothing runs the test suite automatically on every change.
- **Model names are written in files** for most AI features; moving to a newer model means a code change, except for the question bank and the consultant, which read an environment variable.
- **Vendor concentration.** Vercel, Neon and Anthropic each have one point of dependence. The graceful-degradation table in chapter 11 shows what survives losing a service; losing the database stops everything.
- **Email, charts and icons** have no recorded reason for the choice. They are conventional picks.

How to verify any statement in this guide

1. Find the file named in the box (for example `bankModels.ts`) in the repository's `src/lib` folder.
2. Read the comment at the top; written rationales are quoted from there.
3. Run `npm test` to see the rules pass, and open the matching file in `tests/`.

4. For counts (pages, models, migrations, tests) re-run the count on the current code; they will change.

Glossary

API route
A web address the app's code answers with data instead of a page.

Core file
A file of pure rules with no database or network, so it can be tested alone.

Cron job
A task the host runs on a timetable, such as every morning.

Embedding
A list of numbers that stands for the meaning of a text, so two texts can be compared by meaning.

Idempotent
Safe to do twice: the second time changes nothing.

Migration
A reviewed step that changes the database structure.

PWA
Progressive Web App: a website that can be installed on a phone and cope with a poor connection.

RBAC
Role-based access control: what each person may do, decided by the server.

SSE
Server-Sent Events: a one-way stream from the server to the browser.

SSRF
Server-side request forgery: tricking a server into fetching an internal address.

Webhook
A message another service sends to the app when something happens, such as a payment.

zod
A library that checks data has the shape the code expects.

Open items and how to verify this guide

This guide was prepared from the repository without a live system. Use the steps at the end of chapter 13 to check any statement. Counts and model names change as the platform grows; re-check them before quoting them.

Document information and revision history

Item	Detail
Document	AAICBI Ecosystem — Technology and Architecture Guide
Version	1.0
Published	20 October 2026
Prepared for	For leadership, partners and technical reviewers
Basis	The source code, documentation and automated tests of the AAICBI Ecosystem repository on the date above. Behaviour was read from the code; it was not load-tested.
Screenshots	Screenshots in this guide are real screens from a test environment with demonstration accounts and sample data.

Version	Date	Change
1.0	20 October 2026	First edition.

Numbers such as test counts, page counts and thresholds are taken from the repository at the publication date and will change as the platform grows.